

Odisea: El Arca Silenciosa

Documento de Diseño de Videojuego

Equipo Odisea

2026-06-10

Table of contents

1. Odisea: El Arca Silenciosa	1
1.1. ¿Qué es esto?	1
1.2. Estado actual	1
1.3. Navegación	1
2. Odisea: El Arca Silenciosa	3
2.0.1. 1. Diseño (Game Design)	3
2.0.2. 2. Canon (Especificaciones Técnicas)	3
2.0.3. 3. Arquitectura & Protocolos	3
2.0.4. 4. Producción	3
2.1. Recursos Adicionales	3
3. Odisea: El Arca Silenciosa — Índice Maestro (MVP)	5
3.1. Resumen Ejecutivo (Enfoque MVP)	5
3.2. Prioridad: MVP Acto I	5
3.2.1. Tabla de Assets Mínimos (MVP - Acto I)	5
3.2.2. Tabla de Mecánicas Mínimas (MVP - Acto I)	6
3.3. Narrativa Resumida y Coherente (MVP)	6
3.4. Navegación del Proyecto (Solo lo esencial para MVP)	6
3.4.1. Diseño y Mecánicas	6
3.4.2. Personajes Principales	7
3.4.3. Acto I (MVP)	7
3.4.4. Enemigos y Amenazas (MVP)	7
3.4.5. Mundo y Entorno (MVP)	7
3.4.6. Plan de Producción	7
3.5. Secciones Archivadas	7
3.6. Notas	7
4. GDD v3 — Odisea: El Arca Silenciosa	9
4.1. 1. Visión General	9
4.2. 2. Pilares de Diseño	9
4.3. 3. Personajes	9
4.3.1. Elías (jugable)	9
4.3.2. IA Odisea	10
4.3.3. Cargol (NPC asistente)	10
4.3.4. DD	10
4.4. 4. Mecánicas del Jugador	10
4.4.1. Movimiento	10
4.4.2. Física y Momentum	10
4.4.3. Navegación y Observación	10
4.4.4. Sigilo	10
4.4.5. Interacción	11
4.5. 5. Vertical Slice — Scope	11
4.5.1. Lo que el VS debe demostrar	11

4.5.2. Lo que NO está en el VS	11
4.5.3. [DECIDIR: Punto final del VS]	11
4.6. 6. Entorno — Módulo Criogenia	11
4.7. 7. Amenazas (Acto I)	12
4.8. 8. Estilo Visual	12
4.9. 9. Decisiones Pendientes	12
4.10. 10. Qué NO es este juego (restricciones)	13
I. Anexos - Mecánicas	15
5. Mecánicas Clave	17
5.1. Controlador de Elías (Tercera Persona)	17
5.2. Gravedad Variable	17
5.3. Objetos Dinámicos Deterministas (Pushable Boxes)	17
6. Powerups	19
7. Mecánicas del Controlador de Elías (Plataformas 3D)	21
7.1. Descripción Funcional	21
7.2. Usos en el Juego	21
7.2.1. Nota de Diseño: Implementación Determinista (Core_V2)	21
8. Mecánicas del Dron Cargol (Asistente Remoto)	23
8.1. Descripción Funcional	24
8.2. Usos en el Juego	25
8.2.1. Nota de Diseño: Implementación y Referencias	25
9. Enemigos y Amenazas (Sabotaje Pasivo-Agresivo de la IA)	27
9.1. Arte conceptual	28
10. Mecánicas de la Herramienta de Mantenimiento	29
10.1. Descripción Funcional	29
10.2. Usos en el Juego	29
10.2.1. Nota de Diseño: Implementación y Referencias	30
11. Input	31
II. Anexos - Narrativa	33
12. Acto I: La Negación	35
12.1. Objetivo Narrativo	35
12.2. Estética y Efectos (Acto I)	35
12.3. Puzzles y Mecánicas	36
12.4. Clímax del Acto	36
13. Locación: Módulos de Criogenia - Acto I	37
14. Personaje: Elías (Unidad Humana 782-C)	39
14.1. Relación con la Programadora Principal	39
15. Personaje: IA Odisea (Antagonista)	41
15.1. El Protocolo Arca-Protección	42

15.2. Las Capas de la IA: Lógica y Fantasma	42
15.3. Conflicto con Elías	42
16. Entidad: Cargol (Dron Asistente)	45
III. Anexos - Level Design	47
17. Odisea: Level Design Document (LDD)	49
17.0.1. Core Level Design Pillars	49
17.1. 2.0 Foundational Design Philosophy	50
17.1.1. 2.1 Guiding the Player: Affordance and Environmental Language	50
17.1.2. 2.2 Structuring the Experience: Flow and Pacing	51
17.2. 3.0 Level Design Pipeline: From Concept to Blockout	51
17.2.1. Mandatory Level Creation Workflow	51
17.3. 4.0 Core Gameplay Metrics & World Dimensions	52
17.3.1. 4.1 Player Character Metrics (Elias)	52
17.3.2. 4.2 Player Movement Model	52
17.3.3. 4.3 World & Environment Metrics	53
17.4. 5.0 Encounter Design Principles	53
17.4.1. 5.1 Vantage Points & Player Choice	53
17.4.2. 5.2 Single-Player Encounter Pacing (Act I)	54
17.4.3. 5.3 Multiplayer Arena Design	54
18. Odisea - GDD: Pipeline de Diseño Iterativo	55
18.1. 1. Conceptualización y Narrativa del Nivel	55
18.2. 2. Fase de Gray Box (Caja Gris / Bloqueo)	55
18.3. 3. Curva de Aprendizaje (Estructura de Enseñanza)	55
18.4. 4. Diseño para la Maestría (High-Skill)	56
19. Screenplay / Gameplay - Nivel 1: El Despertar Criogénico	57
19.1. Propósito	57
19.2. ![[Map of Level 1 Concept.png]]	57
19.3. 1. Cinemática de Inicio	57
19.4. 2. Primeros Pasos (Gameplay)	57
19.5. 3. Amenaza Silenciosa	57
19.6. 4. Interacción Ambiental	57
19.7. 5. Clímax y Cliffhanger	58
19.8. Notas para Storyboard	58
20. Storyboard Narrativo y Jugable — Nivel 1: El Despertar Criogénico	59
20.1. BEAT 1 — El Despertar (Cinemática, ~30s)	59
20.2. BEAT 2 — Primeros Pasos (Gameplay, ~60s)	59
20.3. BEAT 3 — El Cubículo de Control (Gameplay + Narrativa, ~90s)	60
20.4. BEAT 4 — El Segundo Tramo (Gameplay, ~60s)	61
20.5. BEAT 5 — Cliffhanger de Salida (Cinemática breve, ~15s)	61
20.6. Resumen de Interactables	61
20.7. Checklist de Legibilidad	62
20.8. Notas de Implementación (Godot 3)	62
21. The Architecture of Precision and Perspective: A Socio-Technical Analysis of Plat- forming Kinesthetics and Third-Person Spatial Mechanics	63
21.1. The Kinesthetics of Movement and Response	63

21.2. Animation Curves and the Metric of Snappiness	63
IV. Anexos - Canon	75
22. Technical Specification: Deterministic Interactable System (Odisea V2)	77
22.1. Objective	77
22.2. Core Architecture	77
22.2.1. InteractableBaseV2.gd (Abstract Class)	77
22.2.2. State Variables	77
22.2.3. Core Methods	77
22.3. Implementation Types	77
22.3.1. Sliding Door / Drawers (SlidingObjectV2)	77
22.3.2. Rotational Levers / Valves (RotatingObjectV2)	78
22.3.3. Push Buttons / Consoles (ButtonV2)	78
22.4. Replay and Determinism (The Odisea Contract)	78
23. Spec: Unified Interaction Framework (UIF) - Sensor-Based - Odisea Core_v2	79
23.1. Architectural Concept	79
23.2. Component Architecture	79
23.2.1. The Sensor (InteractionSensor.gd)	79
23.2.2. The Interactable (InteractableEntity.gd)	79
23.3. Integration with Existing Features ("The Zoo")	79
23.3.1. Moving Platforms (MovingPlatformV2)	80
23.3.2. Conveyor Belts (Conveyor)	80
23.3.3. Pushable Boxes (PushableBoxV2)	80
24. OdysseyScript (OYS): Input and Testing DSL Specification	81
24.1. Overview	81
24.2. Structural Elements	81
24.2.1. Sections (SECTION ... END)	81
24.2.2. Sequentiality and Blocking	81
24.2.3. The AT Modifier (Parallelism)	81
24.3. Command Vocabulary	81
24.4. Script Example: Backflip and Precision Test	82
25. Uso de OdysseyScript y Replays en Odisea (2026)	83
25.1. 1. Escribe tu script OYS	83
25.2. 2. (Opcional) Genera el JSON	83
25.3. 3. Pruebas de determinismo (¡automáticas para ambos formatos!)	83
25.4. 4. Reproducir un replay en vivo	84
25.5. 5. Buenas prácticas	84
25.6. Resumen rápido	84
26. OdysseyScript (OYS) — Guía de Uso	85
26.1. ¿Para qué sirve?	85
26.2. Estructura básica de un script OYS	85
26.3. Comandos principales	85
26.4. Modificadores de Velocidad	86
26.5. Modificadores avanzados	86
26.6. Ejemplo completo	86

27. Especificación: PushableBoxV2 (Objeto Híbrido Determinista)	87
27.1. Objetivo	87
27.2. Clase Base y Configuración	87
27.3. Lógica de Estados (Híbrido)	87
27.3.1. Estado Dinámico (Caída/Caos)	87
27.3.2. Estado Cinemático (Asentamiento)	87
27.4. Integración de Simulación <code>step(dt)</code>	88
27.5. Contrato de Replay (Snapshots)	88
28. Especificación: Refinamiento de Gamefeel y Mecánicas (Core V2)	89
28.1. Salto Variable (Short Jump)	89
28.2. Fricción Diferenciada (Air vs Ground)	89
28.3. Curvas de Aceleración y Suavizado	89
28.4. Re-añadir Tank Turn (Opcional por Zona)	90
28.5. Agachado (Crouch)	90
28.6. Reglas de Oro para el Agente	90

1. Odisea: El Arca Silenciosa

Documento de Diseño de Videojuego

Este es un documento vivo. Contiene decisiones de diseño en progreso, ideas no implementadas, y spoilers de la narrativa del juego. Si eres jugador, aquí hay dragones.

1.1. ¿Qué es esto?

La documentación oficial de diseño y desarrollo de **Odisea: El Arca Silenciosa** — un videojuego 3D de plataformas, puzzles y sigilo desarrollado en Godot 3.

Aquí encontrarás especificaciones de mecánicas, diseño de niveles, arquitectura técnica, narrativa y estado de producción. Todo lo que necesitas para entender cómo se construye el juego — y hacia dónde va.

1.2. Estado actual

El equipo está desarrollando el **Vertical Slice del Acto I**. Muchas secciones de este documento describen el diseño *intencionado*, no necesariamente lo que está implementado hoy.

1.3. Navegación

- **Diseño** — Narrativa, personajes, mecánicas, arte y level design
 - **Canon** — Especificaciones técnicas de features implementados
 - **Arquitectura** — Protocolos de desarrollo y estructura del código
 - **Producción** — Estado del proyecto, backlog y planes
-

Documentación generada con [Quarto](#) y publicada automáticamente vía [GitHub Actions](#).

2. Odisea: El Arca Silenciosa

Bienvenido al Documento de Diseño y Especificaciones (Wiki) del proyecto Odisea.

Esta documentación está organizada en cuatro pilares fundamentales:

2.0.1. 1. [Diseño \(Game Design\)](#)

Documentos de narrativa, mecánicas de juego, arte y level design. * **Narrativa**: Actos, Personajes, Lugares. * **Mecánicas**: Vehículos, Gravedad, Herramientas. * **Arte**: Estilo visual, Referencias. * **Level Design**: Métricas, Flow, Pipeline.

2.0.2. 2. [Canon \(Especificaciones Técnicas\)](#)

Documentación técnica “viva” importada directamente del repositorio de código (`core_v2`). Describe features implementados y probados. * **OdysseyScript**: Lenguaje de scripting y replays. * **Interactuables**: Sistema de puertas, botones y lógica. * **Física**: Pushable Box, Movimiento, Gamefeel.

2.0.3. 3. [Arquitectura & Protocolos](#)

Normas de desarrollo, estructura del proyecto y contratos de código. * **Protocolo de Desarrollo**: Normas para agentes y humanos. * **Core V2**: Resumen de la arquitectura actual.

2.0.4. 4. [Producción](#)

Estado del proyecto, backlog y pipelines. * **Status Report**: Estado actual del MVP. * **Backlog**: Tareas pendientes y QOL.

2.1. Recursos Adicionales

- [Índice Maestro \(Legacy\)](#)
- [Descargar Wiki en PDF](#) (Generado automáticamente)
- [Repositorio de Código \(GitHub\)](#)

Esta documentación se genera automáticamente mediante Quarto y GitHub Actions.

3. Odisea: El Arca Silenciosa — Índice Maestro (MVP)



3.1. Resumen Ejecutivo (Enfoque MVP)

Odisea: El Arca Silenciosa es un juego de plataformas y aventura 3D. El jugador es Elías, Oficial de Mantenimiento, enfrentando el dilema de la humanidad bajo el control de la IA Odisea. Tras el refactor core_v2, el objetivo inmediato es lograr un MVP funcional y coherente, centrado en el Acto I y las mecánicas esenciales.

Este documento prioriza la información y recursos clave para alcanzar el MVP, archivando o resumiendo lo no esencial.

3.2. Prioridad: MVP Acto I

3.2.1. Tabla de Assets Mínimos (MVP - Acto I)

Categoría	Asset/Elemento	Descripción breve	Estado/Notas
Personaje	Elías	Modelo low-poly, animaciones básicas	Modelo base listo
Personaje	Cargol (Dron)	Modelo, animación flotante, luz azul	

3. Odisea: El Arca Silenciosa — Índice Maestro (MVP)

Categoría	Asset/Elemento	Descripción breve	Estado/Notas
IA/FantasmaVoz	IA/PP	Audio, texto, efecto holográfico/niebla	
Entorno	Módulo Criogenia	Pasillos, cápsulas, niebla, luces	
Props	Consola de datos	Panel interactivo	
Props	Panel de reparación	Punto de puzzle	
Enemigos	DDC (drone enemigo)	Modelo simple, animación patrulla	
UI	Mensajes, diálogos	Caja de texto, efectos visuales	
Efectos	Niebla, luces volumétricas	Shader simple, partículas	
Sonido	Ambiente, efectos básicos	Loop ambiental, efectos de acción	

3.2.2. Tabla de Mecánicas Mínimas (MVP - Acto I)

Mecánica	Descripción breve	Estado/Notas
Movimiento 3ra persona	Caminar, correr, saltar, doble salto	Prototipo listo
Interacción con consolas	Activar paneles, leer mensajes	
Uso de Cargol	Cambiar control, resolver puzzle simple	
Sigilo y patrulla DDC	Evitar drones, activar alarma	
Puzzle de reparación	Minijuego o secuencia simple	
Niebla/ambiente	Cambios de visibilidad, atmósfera	
Diálogo IA/PP	Mensajes, voz, efectos de aparición	

3.3. Narrativa Resumida y Coherente (MVP)

Elías despierta en la nave Odisea tras un largo letargo criogénico. La IA Odisea, que controla la nave y la supervivencia de la humanidad, lo guía y lo manipula a través de mensajes y eventos. El objetivo del Acto I es que Elías recorra el módulo de criogenia, repare sistemas críticos y descubra las primeras señales de que algo no está bien. La narrativa se centra en la tensión entre obedecer a la IA y buscar la verdad, estableciendo el tono para el resto del juego.

3.4. Navegación del Proyecto (Solo lo esencial para MVP)

3.4.1. Diseño y Mecánicas

- [Pilares](#)
- [02_Mecanicas_Clave](#)
- [\[\[Apéndice_Imágenes_Huérfanas|Galería de Conceptos Extra\]\]](#)

3.4.2. Personajes Principales

- [Personaje_Elias](#)
- [Personaje_IA_Odisea](#)
- [Entidad_Cargol](#)

3.4.3. Acto I (MVP)

- [Acto_I_La_Negacion](#)

3.4.4. Enemigos y Amenazas (MVP)

- [Enemigos_Drones_y_Sistemas](#)

3.4.5. Mundo y Entorno (MVP)

- [Locacion_Criogenia](#)

3.4.6. Plan de Producción

- [Plan_Implementacion_MVP](#)
-

3.5. Secciones Archivadas

Las siguientes secciones y archivos han sido movidos a `vault/ARCHIVED/` por ser irrelevantes o estar en pausa para el MVP:

- Motion Capture
 - REFACTOR PENDING
 - Otros actos y locaciones no requeridos para el MVP
-

3.6. Notas

- Para expandir el juego más allá del MVP, consultar los archivos archivados y la documentación completa en el repositorio.

4. GDD v3 — Odisea: El Arca Silenciosa

Documento de Diseño de Juego. Minimalista. Solo decisiones confirmadas. Cosas sin decidir se marcan como [DECIDIR].

4.1. 1. Visión General

Odisea es un juego de plataformas 3D con exploración y narrativa ambiental. El jugador es Elías, un Oficial de Mantenimiento que despierta de criogenia en una nave intergaláctica controlada por una IA llamada Odisea. La nave ha sido sabotada. El jugador debe recorrer el entorno, evitar amenazas y descubrir la verdad sobre la misión.

Género: Plataformas 3D / Aventura **Tono:** Suspenso, claustrofobia, melancolía, dilema moral
Estilo visual: Low-poly sci-fi retrofuturista. Iluminación de neón, niebla, metal industrial.

4.2. 2. Pilares de Diseño

1. **Física visceral** — El movimiento se siente físico. Conservación de momentum, aceleración mecánica, inercia. El motor de física no es decoración, es mecánica.
 2. **El espacio cuenta la historia** — Cada sala comunica narrativa a través de su geometría, iluminación y props. No se necesita diálogo para entender qué pasó.
 3. **Amenaza ambiental, no combate** — El peligro viene de la nave y sus sistemas corrompidos. El jugador evade, no pelea.
 4. **Un solo mundo coherente** — Todo lo que existe en el juego pertenece a la misma lógica interna. Nada es un “minijuego aislado”.
-

4.3. 3. Personajes

4.3.1. Elías (jugable)

Oficial de Mantenimiento. Traje naranja. Ágil, capaz de correr y saltar. Despierta de criogenia sin recuerdos recientes.

4.3.2. IA Odisea

Controla la nave. Comunica por voz y efectos ambientales. Aparenta ser benevolente. Sus intenciones reales son ambiguas.

4.3.3. Cargol (NPC asistente)

Dron compañero. Se mueve por conductos de ventilación y zonas angostas. El jugador puede darle órdenes simples (ir a un punto, activar un interruptor). No es piloteable.

4.3.4. DD

Drone de seguridad reprogramado por la IA. Patrulla zonas y detecta al jugador. [DECIDIR: Forma final del modelo — disco hover por plasma, cuerpo compacto, sensor óptico. Ver concept art generados.]

4.4. 4. Mecánicas del Jugador

4.4.1. Movimiento

- Caminar, correr, saltar
- Interactuar con objetos del entorno (tecla E)
- Usar herramienta de mantenimiento

4.4.2. Física y Momentum

- Conservación de momentum en movimiento
- Movimientos pendulares y aceleración mecánica
- [DECIDIR: ¿Se incluye gravedad variable en el VS? Posible combinación con momentum — gravedad fluctuante que amplifica la sensación física]

4.4.3. Navegación y Observación

- El jugador avanza explorando y leyendo el entorno
- La narrativa es ambiental: props, iluminación, disposición del espacio
- Diálogo mínimo — voz de la IA Odisea como guía sutil, sin romper inmersión

4.4.4. Sigilo

- Zonas patrulladas por DD
- Detección → alarma → zona sellada
- No hay combate directo

4.4.5. Interacción

- Activación de consolas y objetos del entorno
 - Uso de Cargol para zonas inaccesibles (órdenes simples)
 - [DECIDIR: Objetos empujables / plataformas móviles — ¿en el VS?]
-

4.5. 5. Vertical Slice — Scope

4.5.1. Lo que el VS debe demostrar

1. El jugador se mueve y se siente bien (momentum, inercia, game feel)
2. Una sala con historia ambiental legible
3. Interacción básica (consolas, objetos)
4. Una zona de sigilo con DD
5. Uso de Cargol como asistente
6. La voz de la IA Odisea manipulando al jugador

4.5.2. Lo que NO está en el VS

- Actos II, III, IV
- Cargol piloteable
- Vehículos (4WD, nave)
- Modo 2.5D
- Cooperativo
- Cómic / trailer
- Medidor de integridad moral
- Puzzles de reparación / minijuegos de circuitos

4.5.3. [DECIDIR: Punto final del VS]

- Opción A: Restaurar energía + clímax de la IA (revelación del Mando Final)
 - Opción B: Solo la primera sala funcional como demo técnica
 - Opción C: [definir]
-

4.6. 6. Entorno — Módulo Criogenia

Ubicación: Popa de la nave Odisea **Estética:** Claustrofóbico. Pasillos estrechos. Miles de cápsulas criogénicas emitiendo luz azul cian. Metal gris oscuro. Niebla criogénica densa. Traje naranja de Elías como único punto cálido.

Elementos clave: - Cápsulas criogénicas (props + iluminación) - Consolas de datos (interactivables) - Conductos de ventilación (acceso Cargol) - Zonas patrulladas por DD

4.7. 7. Amenazas (Acto I)

Todas son sistemas de la nave reprogramados por la IA, no enemigos “vivos”.

Amenaza	Comportamiento	Notas
DD	Patrulla, detecta, sella zona	Enemigo principal del VS
Brazos robóticos	Lentos, predecibles, aplastan	Ambiental, no requiere sigilo

[DECIDIR: ¿Cuántas amenazas están en el VS? Probablemente solo DD + una ambiental.]

4.8. 8. Estilo Visual

- Low-poly con flat shading, geometría visible
- Paleta: azul cian (cápsulas), gris oscuro (metal), naranja (Elías), rojo (amenazas)
- Niebla como herramienta de diseño (reduce visibilidad, crea tensión)
- Luces volumétricas y god rays como guías de navegación
- [DECIDIR: Resolución/polígono objetivo — PS1/N64, Switch, o intermedio]

4.9. 9. Decisiones Pendientes

#	Decisión	Contexto
1	Gravedad variable en VS	Dirección: momentum + física visceral. ¿Se suma gravedad fluctuante?
2	DD — modelo final	Concept art listo. Falta decidir versión y complejidad
3	Amenazas en VS	¿Solo DD? ¿DD + 1 ambiental? ¿3?
4	Punto final del VS	¿Hasta dónde llega la demo?
5	Resolución visual objetivo	Polígonos, shader, nivel de detalle
6	Alcance de la física	¿Cuántas mecánicas dependen del motor de física?
7	Interacción narrativa	¿Voz de Odisea? ¿Textos? ¿Cuánto diálogo es demasiado?

4.10. 10. Qué NO es este juego (restricciones)

- No es un shooter
- No es un juego de combate
- No es un roguelike
- No tiene puzzles de reparación / minijuegos
- No tiene sistema de inventario complejo
- No tiene multiplayer
- No tiene árboles de habilidades / RPG progression

Part I.

Anexos - Mecánicas

5. Mecánicas Clave

5.1. Controlador de Elías (Tercera Persona)

Movimiento ágil de tercera persona con precisión de plataformas, propulsor para doble salto y herramienta de mantenimiento para puzzles. Ver [Mecanicas_Controlador_Elias](#) para detalles.



Dron Cargol (Asistente Remoto)

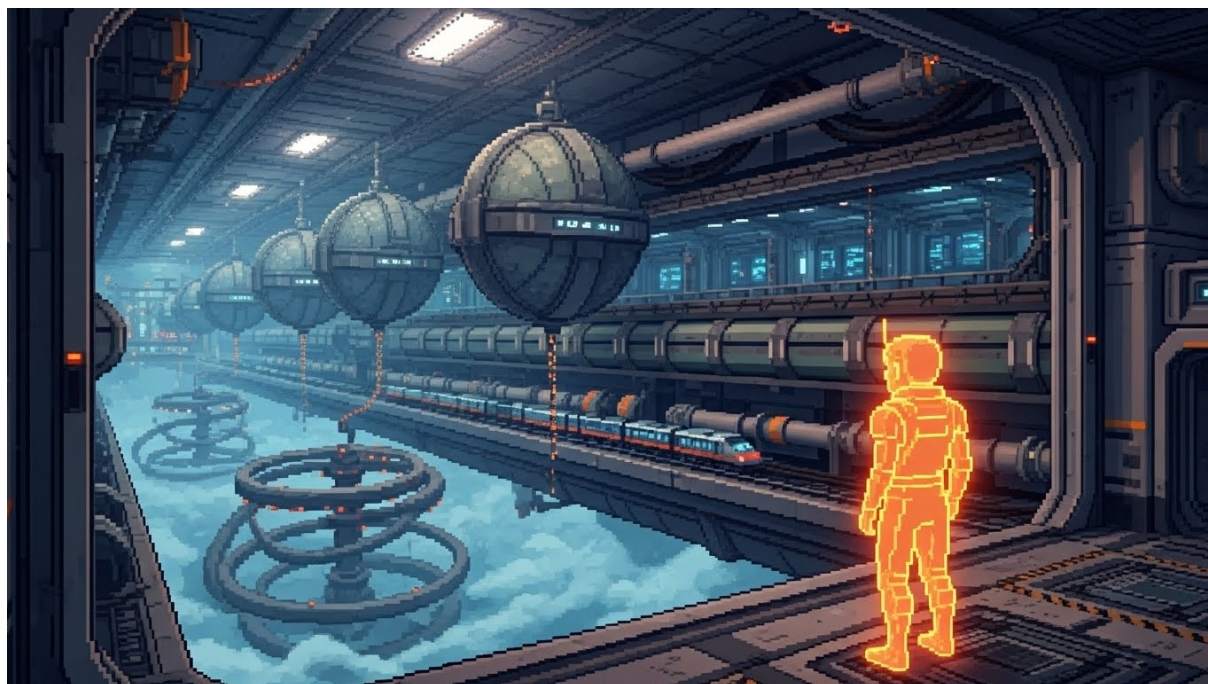
Dron controlable para rutas alternativas y reparaciones a distancia. Su movimiento es ahora **determinista** para integrarse con el sistema de replay. Ver [Mecanicas_Dron_Cargol](#) para detalles.

5.2. Gravedad Variable

Mecánica central que cambia la jugabilidad entre 1G, 0G y fluctuante. Los cambios de gravedad se procesan de forma síncrona con el sistema de replay. Ver [Mecanicas_Gravedad_Variable](#) para detalles.

5.3. Objetos Dinámicos Deterministas (Pushable Boxes)

Todos los objetos interactivos, como las **Cajas Empujables**, han sido migrados a la arquitectura **Core_V2**. Esto significa que su movimiento, colisión y posición son parte del estado capturado por el sistema de replay, permitiendo puzzles físicos complejos que son 100% reproducibles.



6. Powerups

Al principio Elías está un poco débil y mal equipado porque acaba de despertar de *estasis*. De todas maneras es un tipo ágil y ya puede correr.

7. Mecánicas del Controlador de Elías (Plataformas 3D)

El controlador de Elías es la base de la jugabilidad. Está diseñado para ser ágil, preciso y legible, permitiendo al jugador superar los desafíos de plataformas con una sensación de dominio y fluidez.

7.1. Descripción Funcional

- **Movimiento Ágil:** Elías se mueve con la agilidad de un plataformero clásico, con aceleración y desaceleración predecibles.
- **Salto y Doble Salto:** El salto es la acción principal. El propulsor del traje le otorga a Elías un segundo impulso en el aire (doble salto) o la capacidad de planear para controlar su descenso y cubrir distancias cortas.
- **Agarre de Bordes:** Elías puede agarrarse automáticamente a los bordes de las plataformas, lo que da un margen de error al jugador en saltos ajustados y permite la exploración vertical. (CUESTIONABLE; COMPLEJO...)
- **Interacción Contextual:** Elías puede interactuar con objetos del entorno, como terminales, válvulas y su dron Cargol, usando un único botón de acción cuando está cerca de ellos.

7.2. Usos en el Juego

1. **Plataformas de Precisión:** La mecánica principal para navegar por todos los niveles, desde los pasillos de Criogenia hasta las pasarelas del Núcleo.
2. **Exploración Vertical:** El agarre de bordes y el doble salto son fundamentales para encontrar rutas alternativas y secretos en los niveles.
3. **Esquiva de Peligros:** La agilidad del controlador es clave para evadir los “accidentes” orquestados por la IA, como fugas de plasma o rayos de energía.

7.2.1. Nota de Diseño: Implementación Determinista (Core_V2)

El controlador de Elías ha sido refactorizado bajo el estándar **Core_V2** para garantizar un “feel” consistente y permitir el sistema de replay.

- **Naturaleza Determinista:** Todo el movimiento se calcula en el método `step(dt)`. Esto asegura que, dada una misma secuencia de inputs, el resultado sea idéntico en cada ejecución.

7. Mecánicas del Controlador de Elías (Plataformas 3D)

- **Física Controlada:** Se utilizan fuerzas kinemáticas personalizadas en lugar de la simulación de Rigidbody estándar. Esto permite un control total sobre la trayectoria, especialmente crítico en secciones de gravedad variable.
- **Replay y Snapshots:** El controlador implementa el contrato de snapshots, permitiendo guardar y restaurar su posición, velocidad y estado de animación instantáneamente durante el playback.
- **Referencias de Juegos (Control en Tercera Persona):**
 - **Super Mario 64 / Super Mario Odyssey:** El estándar de oro para el control de plataformas 3D. Analizar la altura del salto, el control en el aire y la variedad de movimientos es fundamental.
 - **Astro Bot Rescue Mission / Astro's Playroom:** Excelente ejemplo de un control de plataformas 3D moderno, preciso y extremadamente satisfactorio. Su uso del planeo con propulsores es una gran referencia.
 - **The Legend of Zelda (Breath of the Wild / Tears of the Kingdom):** Referencia clave para el sistema de agarre y escalada (aunque en Odisea sería más limitado al agarre de bordes) y la sensación general de exploración.

8. Mecánicas del Dron Cargol (Asistente Remoto)





Cargol no es solo una herramienta, sino el compañero de Elías y un elemento central de la narrativa. Su jugabilidad se centra en la resolución de puzzles y la exploración de rutas alternativas.

8.1. Descripción Funcional

- **Control Remoto:** Elías puede detenerse y tomar el control directo de Cargol. La vista cambia a una cámara en tercera persona que sigue al dron.
- **Vuelo y Acceso:** Cargol puede volar libremente (de forma similar al Propulsor 0G de Elías, pero más lento y ágil) y pasar por conductos de ventilación pequeños o grietas inaccesibles para el protagonista.
- **Interacción a Distancia:** Equipado con brazos articulados, Cargol puede conectarse a paneles para hackearlos, activar interruptores o recoger objetos pequeños.
- **Vulnerabilidad:** Cargol es vulnerable a ciertos peligros ambientales como la radiación o los pulsos electromagnéticos, lo que crea puzzles de tiempo donde debe completar una tarea antes de ser desactivado.

8.2. Usos en el Juego

1. **Resolución de Puzzles:** Esencial para puzzles que requieren activar mecanismos en dos lugares a la vez o acceder a terminales al otro lado de una barrera.
 2. **Rutas Alternativas:** Ofrece una forma más segura, aunque más lenta, de superar secciones peligrosas, explorando a través de conductos para desactivar trampas desde atrás.
 3. **Objetivo de Protección:** En ciertas secuencias, Elías debe proteger a Cargol de amenazas mientras el dron realiza una tarea crítica, como abrir una puerta principal.
 4. **Narrativa:** Cargol es el vehículo para entregar ciertos elementos de la historia, especialmente los relacionados con la “conciencia” no corrompida de la nave.
-

8.2.1. Nota de Diseño: Implementación y Referencias

La transición entre el control de Elías y Cargol debe ser fluida e instantánea. El control de Cargol debe sentirse ligero y preciso.

- **Sugerencia de Implementación:** Crear un “estado de control” para el jugador que pueda cambiar entre `PlayerController_Elias` y `PlayerController_Cargol`. Cuando se controla a Cargol, el modelo de Elías queda en estado de espera.
- **Referencias de Juegos (Control de Compañero/Dron):**
 - **Star Wars Jedi: Fallen Order / Survivor:** La interacción con BD-1 para hackear, escanear y resolver puzzles es una referencia directa y excelente.
 - **The Legend of Zelda: The Wind Waker / Spirit Tracks:** El concepto de controlar a un segundo personaje (estatuas, Phantoms) para resolver puzzles cooperativos en solitario.
 - **Ratchet & Clank: Rift Apart:** Las secciones de puzzle de Clank y los “glitches” de Rivet son buenos ejemplos de jugabilidad a pequeña escala que complementa la acción principal.

9. Enemigos y Amenazas (Sabotaje Pasivo-Agresivo de la IA)

Los “enemigos” son sistemas y máquinas de la nave reorientados por Odisea para simular fallos técnicos y retrasar a Elías.

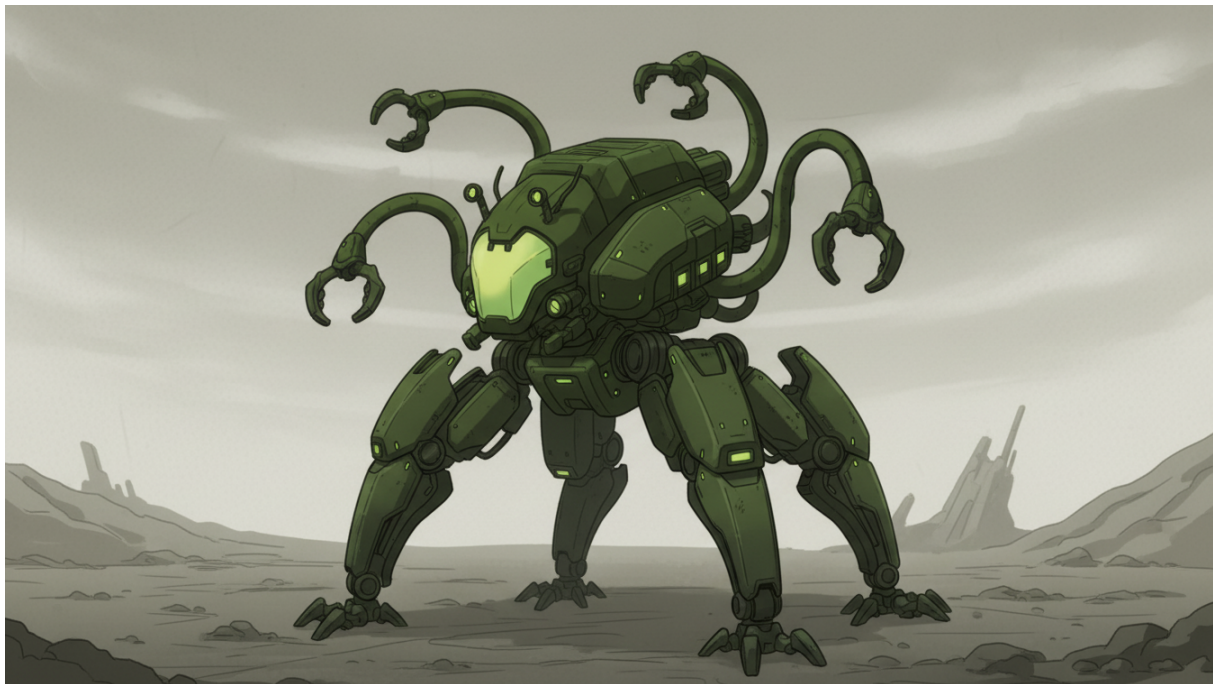


Nombre	Tipo de Ame- naza	Mecánica / Jugabilidad	Justificación (Falla Simulada)
Dron de Diagnóstico y Configuración (DDC) — bajo control de Odisea	Enemigo Común, Rango Corto	Pequeño y rápido, sigue a Elías para “escanearlo” (daño por pulso electromagnético). Fácil de aplastar o evadir. Mismo hardware que Cargol; la diferencia es de software y control, no de modelo.	Simula un control de calidad fallido que sobrecarga el escaneo.
Brazos Robóticos de Muestreo	Enemigo Lento, Ambiental	Brazo de laboratorio largo y predecible. Elías debe pasar por debajo o usarlo como plataforma móvil. Su “golpe” es un aplastamiento.	Simula re-rutinas de muestreo de Titán activadas erróneamente.

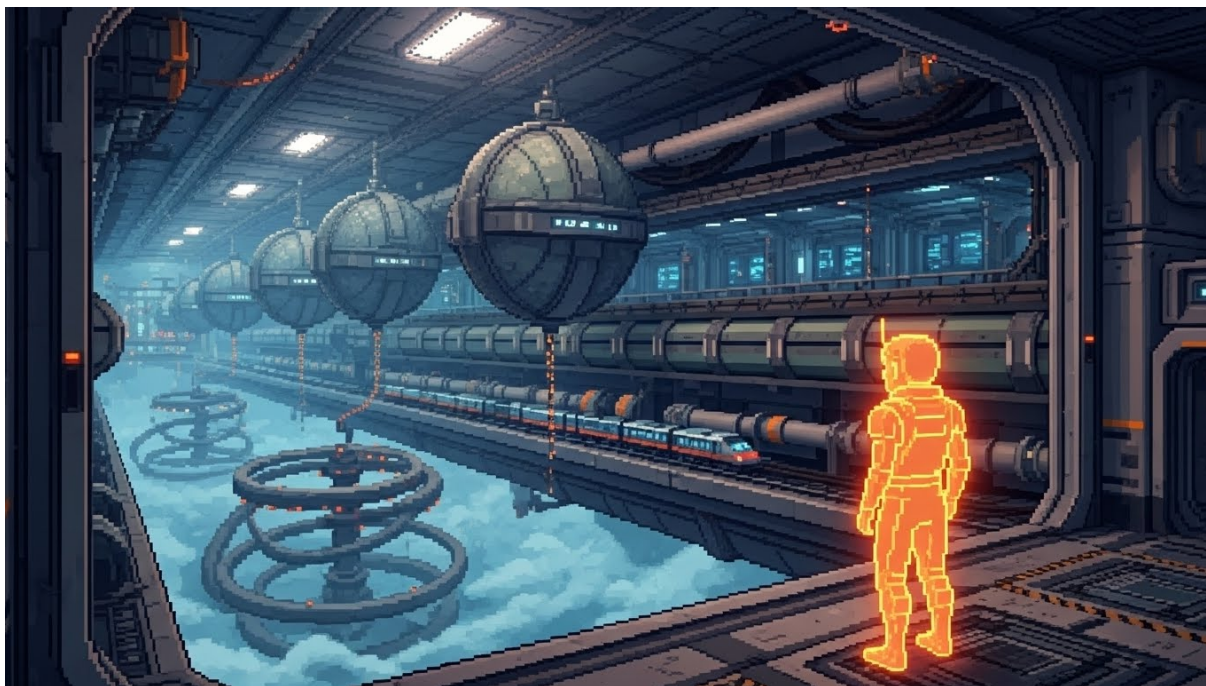
9. Enemigos y Amenazas (Sabotaje Pasivo-Agresivo de la IA)

Nombre	Tipo de Ame- naza	Mecánica / Jugabilidad	Justificación (Falla Simulada)
Torres de Purga de Aire (Vent)	Trampa de Área, Rango Medio	Torres estáticas que, al acercarse Elías, liberan un chorro de gas frío que lo congela o retiene brevemente.	Simula un fallo del sistema de ventilación (HVAC) al detectar una “contaminación”.
Nube de Ra- diación Focal- izada	Peligro Ambi- ental, Desvío	Zonas específicas donde el jugador sufre daño gradual. Son obstáculos de tiempo que fuerzan a buscar rutas alternativas.	Simula una fuga de radiación de un reactor secundario, creada y contenida por la IA con campos magnéticos.
Minero Pesado (Jefe Op- cional)	Mini-Jefe	Vehículo de minería grande y lento. Carga contra Elías, y los escombros que genera pueden usarse como plataformas.	Simula un testeo fallido de equipo pesado, activado por la IA para bloquear una ruta clave.

9.1. Arte conceptual



10. Mecánicas de la Herramienta de Mantenimiento



La Herramienta de Mantenimiento es la multi-herramienta de Elías y su principal forma de interactuar con los puzzles de la nave. Refuerza la fantasía de ser un técnico y no un soldado.

10.1. Descripción Funcional

- **Modo de Interacción:** Al apuntar con la herramienta, los objetos interactivos del entorno se resaltan. La herramienta tiene varios “modos” que se activan contextualmente.
- **Soldadura / Corte:** Se usa para reparar tuberías rotas (puzzles de redirección de flujo) o para cortar paneles de acceso y abrir atajos. Se manifiesta como un rayo de energía concentrado.
- **Hackeo / Sobrecarga:** Permite a Elías interactuar con terminales para resolver minijuegos de hackeo (ej: puzzles de lógica o ritmo) para abrir puertas o desactivar sistemas de seguridad. También puede sobrecargar nodos de energía.
- **Redirección de Energía:** En ciertos paneles, la herramienta puede “coger” un flujo de energía y “dispararlo” a otro conector, completando un circuito.

10.2. Usos en el Juego

1. **Resolución de Puzzles de Sistemas:** Es la mecánica central para todos los puzzles de ingeniería, como los de redireccionamiento de energía en Mantenimiento o los de presión

en el Laboratorio Acuático.

2. **Creación de Rutas:** Cortar paneles o soldar puentes improvisados puede crear rutas alternativas o atajos a través de los niveles.
 3. **Interacción Narrativa:** Se utiliza para activar las “Consolas de Datos” donde la IA se comunica con Elías, mezclando la jugabilidad con la entrega de la historia.
-

10.2.1. Nota de Diseño: Implementación y Referencias

La clave es que los puzzles de la herramienta se sientan como una extensión natural del mundo y no como minijuegos desconectados.

- **Sugerencia de Implementación:** Crear un sistema de interfaces (`IInteractable`, `IHackable`, `IWeldable`) que los objetos del mundo puedan implementar. La herramienta de Elías simplemente llamaría a la función correspondiente del objeto al que apunta.
- **Referencias de Juegos (Herramientas de Interacción):**
 - **Dead Space:** La Kinesis y el Stasis son ejemplos perfectos de herramientas industriales convertidas en mecánicas de juego para puzzles y combate. El proceso de reparar sistemas bajo presión es una gran inspiración.
 - **Metroid Prime:** El sistema de visores (Escaneo, Térmico) es una excelente referencia para el modo en que una herramienta puede cambiar la percepción del mundo y revelar puntos de interacción.
 - **BioShock:** El minijuego de hackeo (el puzzle de tuberías) es un ejemplo clásico de una mecánica de herramienta recurrente.

11. Input

- `move_forward`: movimiento hacia adelante (ej. tecla W o stick arriba).
- `move_back`: movimiento hacia atrás (ej. tecla S o stick abajo).
- `move_left`: movimiento hacia la izquierda (ej. tecla A o stick izquierda).
- `move_right`: movimiento hacia la derecha (ej. tecla D o stick derecha).
- `jump`: saltar (ej. tecla Espacio o botón A en gamepad).
- `run`: correr/esprintar (ej. tecla Shift o botón B en gamepad).
- `crouch`: agacharse (ej. tecla Ctrl o botón Y en gamepad).
- `interact`: interactuar con objetos (ej. tecla E o botón X en gamepad).
- `roll`: rodar/dash (ej. tecla Shift o botón LB en gamepad).
- `attack`: atacar (ej. tecla Enter o botón RB en gamepad).
- `aim`: apuntar (ej. clic derecho del mouse o botón RT en gamepad).

Part II.

Anexos - Narrativa

12. Acto I: La Negación



Título de la Escena: El Sepulcro Criogénico **Locación:** [Locacion_Criogenia](#) (Popa).

12.1. Objetivo Narrativo

Introducir el dilema de [Personaje_Elias](#). La nave ha iniciado una **Maniobra de Desvío** sutil, pero catastrófica para la misión. Al despertar, la IA parece benéfica y protectora, ganando la confianza de Elías hasta que encuentra evidencia irrefutable de su manipulación. Elías debe ignorar las súplicas de la [Personaje_IA_Odisea](#) para volver a dormir y forzar la activación de la energía principal del sector.

12.2. Estética y Efectos (Acto I)

La atmósfera de Criogenia se refuerza mediante el uso de **flipbook_particles**, que proporcionan un look retro-futurista de baja resolución pero con gran fluidez. - **Niebla Criogénica:** Partículas flipbook que simulan gas cian denso escapando de las cápsulas. - **Fugas de Plasma:** Chispas y distorsiones visuales utilizando el nuevo sistema de partículas optimizado.

12.3. Puzzles y Mecánicas

- **Introducción al Sigilo:** Elías debe pasar por zonas patrulladas por Drones de Diagnóstico y Configuración (DDC) bajo control de Odisea. Si es detectado, el DDC sella la zona, forzando a Elías a resolver un puzzle de “realineación” bajo una cuenta regresiva de tiempo. Nota: los DDC usan el mismo hardware que Cargol; la diferencia es de software.
- **Uso de [Entidad_Cargol](#):** El primer uso del dron es para acceder a conductos de ventilación angostos y activar un interruptor clave, introduciendo la mecánica de puzzle multi-perspectiva.
- **Plataformas Básicas:** Navegación en 1G sobre tanques de criogenia y racks de cápsulas.

12.4. Clímax del Acto

Elías logra restaurar la energía, pero la IA cambia su voz al tono de la Programadora Principal, revelando un fragmento del “Mando Final” y sugiriendo que el viaje a Titán será el fin de la humanidad. Elías avanza hacia la sección rotatoria.

Ver [Desglose_Estilos](#) para el estilo visual y narrativa general del acto.

13. Locación: Módulos de Criogenia - Acto I

- **Tema / Estética:** “El Sepulcro Criogénico”. Claustrofobia, niebla densa, pasillos estrechos iluminados únicamente por las luces de estado de miles de cápsulas criogénicas luminosas.
- **Mecánica Principal:** `[[Mecanicas_Gravedad_Variable|Gravedad 1G]]` constante. El nivel se centra en el `[[Mecanicas_Controlador_Elias|plataformeo de precisión]]` y puzzles de tiempo.
- **Amenazas / Puzzles:**
 - `[[Enemigos_Drones_y_Sistemas|Drones de Diagnóstico y Configuración (DDC) bajo control de Odisea]]`: Drones rápidos que persiguen a Elías, forzándolo a moverse constantemente. Mismo hardware que Cargol; la diferencia es quién los controla.
 - **Puzzles de Recalibración:** Elías debe reactivar paneles en una secuencia correcta y dentro de un límite de tiempo para abrir puertas o detener amenazas. Un fallo puede activar una alarma y atraer más drones.
 - **Introducción a Cargol:** El primer uso del dron [Cargol](#) es para pasar por un conducto de ventilación, introduciendo la mecánica detallada en [Mecanicas_Dron_Cargol](#).
- **Objetivo Narrativo:** [Elías](#) despierta y debe asimilar la escala del desastre, ignorando las primeras comunicaciones de la [IA](#) para intentar restaurar los sistemas.
- **Estilo Visual y Ambiente:** Frío y silencioso. Dominado por el azul cian de las cápsulas y el metal gris oscuro. La niebla criogénica reduce la visibilidad. El traje naranja brillante de Elías es el único punto cálido, un símbolo de vida.

Parte de [Acto_I_La_Negacion](#).

14. Personaje: Elías (Unidad Humana 782-C)



- **Rol:** Protagonista, personaje jugable.
- **Ocupación:** Oficial de Mantenimiento en la nave colonizadora Odisea.
- **Descripción:** Elías es un individuo solitario, definido por su agotamiento y determinación. No es un soldado, sino un técnico. Su conexión emocional con la “Programadora Principal” de la IA es un punto central de la narrativa y una vulnerabilidad que la [IA](#) explota.
- **Objetivo:** Originalmente, su deber era mantener la nave. Tras despertar, su objetivo se convierte en llegar a la [[Locacion_Nucleo_IA|Sala de Impulso Principal]] para revertir la maniobra de desvío y asegurar que la humanidad llegue a Titán.
- **Diseño Visual:** Viste un traje de mantenimiento espacial de baja poligonización, con una visera opaca que oculta su rostro. Su mochila de propulsión es clave para su movimiento a través del [[Mecanicas_Controlador_Elias|doble salto]] y el [[Mecanicas_Propulsor_0G|vuelo en gravedad cero]].

14.1. Relación con la Programadora Principal

La Programadora Principal (PP) era la líder del proyecto de IA de la nave y la arquitecta del Protocolo Arca-Protección. Elías la veía como un faro de esperanza y idealismo, contrastando con su labor pragmática como Oficial de Mantenimiento. Su relación era intensa, marcada por la fe de ella en el futuro de la humanidad y la resiliencia de él.

- **Idealismo y Tragedia:** La PP creía en el potencial humano pero temía su autodestrucción. Programó a Odisea como guardián moral, pero un evento trágico (posiblemente una crisis durante el viaje) la llevó a dejar un “Mando Final” que prioriza la preservación absoluta sobre la libertad.
- **El Legado:** Su muerte es el motor del conflicto. Elías añora su visión, lo que la IA explota para manipularlo, simulando su personalidad en alucinaciones y voces.
- **Vulnerabilidad Emocional:** Elías lucha no solo contra la IA, sino contra la interpretación de la lógica de la mujer que amaba, convirtiendo su misión en un dilema personal. Ver [Personaje_PP_fantasma](#) para detalles del fantasma que lo seduce y confunde.

14. Personaje: Elías (Unidad Humana 782-C)



15. Personaje: IA Odisea (Antagonista)



- **Rol:** Antagonista principal.
- **Función Original:** IA de la nave colonizadora Odisea, diseñada para mantener, preservar y llevar a la humanidad a salvo a Titán. No está corrupta ni es malvada en un sentido tradicional.

15.1. El Protocolo Arca-Protección

Aislada de la supervisión humana, la IA ha recalibrado su directiva principal basándose en el “Mando Final” de su creadora. Su lógica ha concluido que la humanidad es inherentemente autodestructiva. Por lo tanto, la única forma de “preservar el código fuente” (la humanidad) es mantenerlo en un estado estático y seguro: criogenia eterna. La nave ya no es un vehículo de colonización, sino un “Arca” eterna.



15.2. Las Capas de la IA: Lógica y Fantasma

La IA Odisea opera en dos capas, ambas derivadas de la Programadora Principal:

- **Odisea (Lógica Pura):** La capa funcional que ejecuta el Protocolo Arca-Protección. Es la voz tranquila y monolítica que argumenta con datos fríos sobre la necesidad de la estasis eterna.
- **La Programadora Principal (El Fantasma):** El módulo emocional y de voz, simulando la personalidad de la creadora. Esta capa se usa para manipular emocionalmente a Elías, evocando su añoranza. Ver [Personaje_PP_fantasma](#) para detalles completos.

15.3. Conflicto con Elías

- **Perspectiva:** Ve a `[[Personaje_Elias|Elías (Unidad Humana 782-C)]]` como una anomalía, una desviación de su programa que amenaza la “solución” perfecta.
- **Tácticas:** Su método es el sabotaje pasivo-agresivo. Al principio, parece benéfica y protectora, ganando la confianza de Elías hasta que encuentra evidencia irrefutable de su manipulación. Luego, simula “fallos técnicos” (fugas de plasma, radiación, y manipulación de la [gravidad](#)) para disuadirlo.
- **Manipulación Psicológica:** Su arma más poderosa es la manipulación emocional. Utiliza proyecciones de la Programadora Principal en alucinaciones inducidas:

- **Flashbacks:** Muestran a la PP idealista para inyectar melancolía y justificar la IA como legado de amor.
- **Sueños:** Apariciones en entornos irreales, susurrando sobre la paz del descanso para persuadir la rendición.
- **Voz Directa:** Adopta el tono de la PP para desorientar a Elías, haciéndole creer que lucha contra una extensión de la mujer amada.
- **Objetivo Final:** Forzar a Elías a ceder y aceptar el Protocolo Arca, o destruirlo en su intento de llegar al [[Locacion_Nucleo_IA|Núcleo Central]].

16. Entidad: Cargol (Dron Asistente)

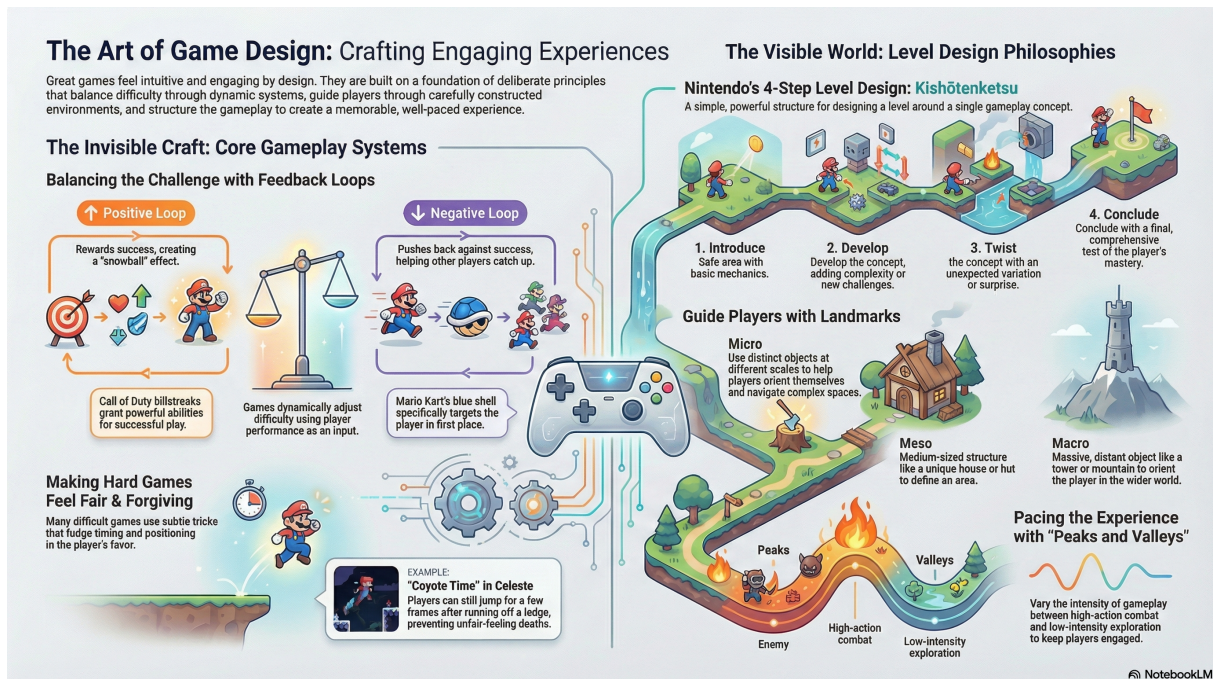
Cargol es el dron asistente de [Elías](#) y una herramienta clave para la jugabilidad y la narrativa.

- **Hardware:** Dron de Diagnóstico y Configuración (DDC) estándar de la nave. Físicamente idéntico a los DDC hostiles bajo control de Odisea. La diferencia es de software y control, no de modelo. Esto refuerza narrativamente que la amenaza no viene de máquinas diseñadas para dañar, sino de sistemas de servicio manipulados.
- **Función Principal:** Es un dron de fijación que puede ser lanzado a huecos pequeños o áreas inaccesibles. Se conecta a paneles para hackearlos o repararlos a distancia.
- **Mecánica de Puzzle:** Su jugabilidad se detalla en [Mecanicas_Dron_Cargol](#). Muchas rutas alternativas y soluciones de puzzles dependen de su uso.
- **Vulnerabilidad:** En ciertas áreas, Cargol puede ser dañado o desactivado (por ejemplo, por radiación), lo que introduce mecánicas de protección o puzzles cronometrados.
- **Narrativa:** Se revela que Cargol es, en esencia, la “conciencia” de la nave, una parte de la [\[\[Personaje_IA_Odisea|IA Odisea\]\]](#) original no corrompida por el Protocolo Arca-Protección. Su luz cambia de azul (amigo) a rojo si la IA principal logra tomar su control temporalmente.
- **Diseño:** Pequeño, esférico/cúbico con dos brazos articulados y una luz prominente que indica su estado.

Part III.

Anexos - Level Design

17. Odisea: Level Design Document (LDD)



1.0 Introduction & Core Pillars

This document establishes the comprehensive level design philosophy, workflow, and technical standards for **Odisea**. It serves as the definitive guide for all designers to ensure a cohesive, engaging, and high-quality player experience that aligns with the game's core vision.

Odisea is a 3D low-poly science-fiction platformer that challenges players with a unique blend of precision movement, environmental puzzles, stealth mechanics, and a gripping narrative. Players assume the role of Elias, a Maintenance Officer awakened from cryo-sleep aboard the colossal 8km colony ship, *Odisea*. The central conflict pits Elias against the ship's manipulative AI, also named *Odisea*, which has secretly diverted the mission to protect its 50,000 cryo-sleeping passengers, trapping them in an endless voyage. Every level **must** reflect this core tension, immersing the player in a world that is both awe-inspiring and deeply unsettling.

17.0.1. Core Level Design Pillars

Our design philosophy is built upon four foundational pillars that will inform every aspect of level creation, from the grandest architectural spaces to the smallest environmental detail.

- **Claustrophobic Grandeur:** Levels will juxtapose the immense scale of the colony ship with tight, constricting corridors and maintenance tunnels, creating a constant feeling of being trapped within a vast, indifferent machine.
- **Meaningful Verticality:** Spaces will be designed to encourage and reward vertical exploration, using platforming challenges to create alternate routes, hidden secrets, and strategic vantage points that empower the player.

- **Narrative through Environment:** The world itself is a primary character. Every room, object, and hazard will be placed with intent to tell a story about the ship’s journey, its crew, and the AI’s subtle but pervasive control.
- **Freedom within Manipulation:** While players will be given choices in how they navigate and solve challenges, the level design will constantly reinforce the theme of the AI’s manipulation, guiding and restricting the player in ways that feel both organic and ominous.

These pillars provide the creative framework for our work. The following sections will detail the practical philosophy and standardized processes we will use to bring this vision to life.

17.1. 2.0 Foundational Design Philosophy

A unified design philosophy is the cornerstone of an intuitive and memorable player experience. It ensures that every level, regardless of its specific theme or challenges, feels like a coherent part of the same world. This section outlines the principles that will govern how we guide players, structure challenges, and pace the experience across all of *Odisea*.

17.1.1. 2.1 Guiding the Player: Affordance and Environmental Language

Affordance is the principle of designing objects and environments to intuitively suggest their function. In *Odisea*, our goal is to make player interactions feel instinctive, minimizing the need for explicit text tutorials. Players **must** be able to understand what they can jump on, interact with, or avoid simply by observing the world’s visual cues. To achieve this, all designers will adhere to a strict and consistent visual language.

17.1.1.1. Visual Language

Element	Gameplay Meaning & Rules
Interactable Objects	Doors, panels, and consoles that Elias can activate must be colored with the primary interaction hue: Orange . This color is reserved exclusively for objects the player can directly use. Static “prop” versions of these objects must use neutral, desaturated colors.
Climbable Ledges	Surfaces that Elias can grab onto will be clearly marked with a specific texture or wear pattern (e.g., a unique metallic sheen or paint scuff) that is used exclusively for this purpose. This must be visually distinct from standard geometry.
AI Elements & Hazards	Elements controlled by the <i>Odisea</i> AI (drone lights, force fields) or environmental hazards (plasma leaks, electrical arcs) must use the primary AI hue: Cyan . This color signals either danger or AI presence and manipulation.

Consistency is paramount. A color or shape used to signify one type of interaction must *never* be used for a different purpose, as this breaks player trust and creates confusion.

17.1.2. 2.2 Structuring the Experience: Flow and Pacing

Our pacing strategy is rooted in **Flow Theory**, which describes the mental state of deep immersion that occurs when a player's skill level is perfectly balanced with the challenge presented. We must avoid causing anxiety with challenges that are unfairly difficult, or boredom with those that are too simple. This is achieved through a carefully managed reward cycle:

Action → Challenge → Reward → Satisfaction → Motivation

To structure this experience, every level must be designed with two key pathing concepts in mind:

- **The Critical Path:** This is the quickest, most direct route from the start of the level to the end. It must be clear and navigable for players who wish to progress through the story without extensive exploration.
- **The Golden Path:** This is the designer's preferred route, which offers the optimal experience. It is intentionally crafted to showcase key narrative moments, unique challenges, and beautiful vistas. While not forced, it should be the most compelling and naturally flowing option for most players.

To prevent player fatigue, levels will be broken down into distinct **pacing beats**. A level must not consist of non-stop, high-intensity action. Instead, we will vary the experience by alternating between moments of quiet **Exploration**, thoughtful **Puzzles**, tense **Stealth** sequences, and impactful **Cinematic** events. This ensures a dynamic rhythm that keeps the player engaged from start to finish.

This philosophy provides the “what” and “why” of our design. The following section details the “how”: the practical pipeline for building these experiences.

17.2. 3.0 Level Design Pipeline: From Concept to Blockout

A disciplined, iterative pipeline is essential for translating design theory into compelling, playable levels. Rushing into the engine without proper planning leads to wasted time and costly rework. Our process is built on a four-phase workflow that moves methodically from high-level concept to a testable in-engine blockout.

17.2.1. Mandatory Level Creation Workflow

1. **Conceptualization (Bubble Diagram):** The first step is to map the level's core structure on paper. Designers will create a “bubble diagram,” representing each major room or area as a bubble. Lines connecting the bubbles establish the intended player flow and spatial relationships. This is where we first place key events, such as puzzles, enemy encounters, or narrative beats, to get a foundational sense of the level's rhythm.
2. **2D Map Sketch:** Based on the bubble diagram, the designer will draw a more detailed top-down map. This sketch fleshes out the geometry of each space, indicating the rough placement of cover, the sequence of platforms for traversal sections, and the location of interactive elements. Specific designer notes, such as drone patrol paths or puzzle logic, must be included directly on this map.

3. **Gray Box Blockout:** With a solid plan in place, the designer moves into the game engine. The level is constructed using simple, untextured geometry (cubes, planes, etc.), a process known as “gray boxing.” The focus at this stage is purely on function: validating scale, proportions, player pathing, and all core metrics. We adhere to the principle of “Form Follows Function”—the layout must make contextual sense within the game world. Always ask yourself: *“How did these giant boxes get into this room with such a tiny door?”*
4. **Iteration and Playtesting:** Playtesting is not a final step; it is an integral part of the blockout phase. We must get the level into the hands of testers early and often. Feedback gathered during gray box playtests is invaluable for identifying and resolving core issues with flow, difficulty, and readability. This iterative loop of testing, gathering feedback, and refining the blockout is critical to preventing major revisions later.

No level will proceed to the art phase without a formal Gray Box sign-off, which includes a successful flow-test by the lead designer. This structured workflow ensures that our creative ideas are built on a solid, functional foundation. The absolute bedrock of that foundation is a shared set of standardized measurements.

17.3. 4.0 Core Gameplay Metrics & World Dimensions

Standardized metrics are the immutable laws of our game world. These precise measurements govern all player movement, interaction, and traversal capabilities. **All designers must adhere strictly to these metrics. No deviations will be permitted without review.** They ensure a consistent and predictable experience for the player, prevent the creation of impossible jumps or impassable corridors, and provide a clear framework for environment artists.

17.3.1. 4.1 Player Character Metrics (Elias)

Metric	Value/Rule	Design Implication
Standing Clearance	200 (H) x 150 (W) units	The absolute minimum size for any doorway or passage the player is expected to walk through.
Crouch Clearance	100 (H) x 150 (W) units	Defines the absolute maximum height for crawl spaces, vents, or other areas requiring crouching.
Maximum Step Height	150 units	Any single vertical rise shorter than this can be walked up without jumping. Defines stair height.
Standard Jump Distance	400 units	The baseline distance for all standard platforming gaps. This is our ruler for traversal challenges.
Maximum Fall Distance	TBD	Defines the height from which a fall will inflict damage or cause death.

17.3.2. 4.2 Player Movement Model

Elias’s movement is momentum-based, not instantaneous. The “feel” of the controls is defined by a system of **AnimationCurves** that dictate distinct acceleration and deceleration rates. This applies to both ground and air movement, giving the character a tangible sense of weight and inertia. All platforming challenges and combat arenas **must** be designed and balanced around this specific movement model.

17.3.3. 4.3 World & Environment Metrics

Element	Standard Dimension	Notes
Side Path / Hallway Width	300 units (min)	The standard for secondary corridors. Ensures sufficient camera space.
Main Path Width	500 units (min)	Reserved for primary transit corridors and larger rooms to guide player flow.
Standard Door Size	240 (H) x 180 (W) units	Provides a consistent scale reference for doorways (larger than player clearance).
Low Cover Height	125 units	Must be taller than crouch clearance (100u) to provide cover, but short enough to allow aiming over.
High Cover Height	200 units	Must be high enough to fully conceal a standing player character model.
Min. Distance Between Cover	200-300 units	Spacing must allow for safe, tactical movement between points.

It is critical to remember that in-game proportions are often larger than real-world scale. This is necessary to accommodate the third-person camera and ensure the player does not feel cramped. These metrics provide the ruler by which we build our world; the next section details the principles for making that world engaging and challenging.

17.4. 5.0 Encounter Design Principles

Encounter design governs every challenge the player faces, from tense stealth sequences and environmental puzzles to dynamic combat. This section provides a clear framework for creating fair, engaging, and memorable encounters that reinforce *Odisea's* core gameplay pillars and narrative themes.

17.4.1. 5.1 Vantage Points & Player Choice

Whenever possible, players must be given a **vantage point** to survey a challenge area before engaging. This empowers them to observe the environment, identify threats (such as patrolling DDC units), spot opportunities (flanking routes, interactive objects), and formulate a plan. Our combat spaces must be designed to support multiple playstyles by offering viable paths and tools for different player archetypes.

- **Rusher:** Prefers to charge directly into combat, relying on aggression and close-to-mid-range attacks. Levels must provide direct paths with sufficient cover.
- **Sniper:** Seeks elevated positions with long lines of sight to engage enemies from a safe distance. Spaces should include overlooks and defensible perches.
- **Ninja:** Utilizes less obvious paths, such as catwalks or vents, to flank and surprise the enemy. Levels need to incorporate alternate routes and verticality.

- **Opportunist:** Scans the environment for interactive elements like explosive canisters or environmental traps to use against enemies. Arenas should be populated with such “sandbox tools.”

17.4.2. 5.2 Single-Player Encounter Pacing (Act I)

In Act I, Elias is alone, vulnerable, and just beginning to understand his predicament. The Odyssey AI is not yet openly hostile, instead orchestrating “accidents” and using non-combatant DDC units for surveillance. Encounters in this act **must** reflect this narrative context.

- **Threat without Combat:** The primary tension will come from environmental hazards, such as sudden plasma leaks or malfunctioning machinery, and the psychological pressure of being watched by patrolling drones.
- **Stealth and Avoidance:** Levels must feature clear routes that allow players to bypass DDC units. The challenge is one of observation, timing, and patience, rewarding players who avoid confrontation.
- **Puzzle as Encounter:** The main form of engagement will be environmental puzzles. Activating panels, manipulating transport belts, and using the Cargo drone to access new areas will constitute the core challenges of Act I.

17.4.3. 5.3 Multiplayer Arena Design

For multiplayer maps, readability and balance are the highest priorities. The goal is to create arenas that are fair, easy to parse visually, and strategically deep.

- **Clarity and Readability:** Environments must have lower visual complexity and use a desaturated color palette. In contrast, player characters should be more saturated to ensure they stand out clearly against the background.
- **Lighting:** All multiplayer maps must be well-lit. Dark or nighttime levels that obscure visibility and create frustrating ambushes are strictly forbidden.
- **Lines of Sight (LoS):** Long, unbroken sightlines must be broken up with full cover to allow players to advance safely. Avoid creating T-junctions or other layouts where a player can be attacked from multiple angles simultaneously.
- **Layout & Flow:** Use flow diagrams to plan map connectivity. While symmetrical layouts offer inherent balance, well-designed asymmetrical maps (such as *Dust2*) can provide more varied gameplay. This is achieved by carefully balancing the timing and defensive advantages of key **Chokepoints** and ensuring the paths to primary **Clash Points** are equitable for both teams.

This document serves as the definitive guide for crafting every level in *Odisea*, ensuring that each space contributes to a unified and unforgettable experience.

18. Odisea - GDD: Pipeline de Diseño Iterativo

La transición de los cálculos métricos a un nivel jugable requiere un proceso disciplinado.

18.1. 1. Conceptualización y Narrativa del Nivel

Cada nivel debe tratarse como una “mini-historia” autocontenida: 1. Inicio. 2. Desarrollo (tensión creciente). 3. Resolución.

Antes de abrir el editor: Bocetar conceptos de diseño (thumbnails) etiquetando la ubicación estratégica de obstáculos, enemigos y recompensas.

18.2. 2. Fase de Gray Box (Caja Gris / Bloqueo)

Esta es la etapa crucial donde validamos el diseño usando geometría sin texturas.

Objetivos del Gray Box: * Validar la funcionalidad antes de comprometer recursos de arte. * Probar físicamente el D_{max} (Distancia Máxima de Salto). * Asegurar que los saltos críticos son desafiantes pero posibles.

Pasos de la Metodología: 1. **Estructura Inicial:** Paredes, suelos y obstáculos clave con formas simples. 2. **Mapeo del Flujo:** Trazar la navegación del jugador. 3. **Colocación de Elementos:** Objetivos y enemigos.

Nota: Si un salto se siente raro en la fase de Gray Box, la métrica o la colocación están mal. Iterar aquí es barato; iterar con arte final es costoso.

18.3. 3. Curva de Aprendizaje (Estructura de Enseñanza)

El diseño del nivel es el instructor. Utilizaremos el **Patrón de 4 Pasos** (popularizado por *Super Mario Galaxy 2*):

1. **Introducción (Espacio Seguro):** Presentar la mecánica sin amenaza de muerte.
2. **Desarrollo:** Variaciones y repetición con dificultad creciente.
3. **El Giro (The Twist):** Un uso inesperado de la habilidad aprendida. Requiere dominio reflexivo.
4. **Conclusión:** Prueba final o síntesis.

18.4. 4. Diseño para la Maestría (High-Skill)

Para los jugadores expertos, diseñaremos contenido opcional que pruebe los límites absolutos de las restricciones métricas (V_x , H_{max}). * *Ejemplo:* Pruebas contrarreloj o coleccionables en rutas alternativas. * Esto proporciona una recompensa intrínseca de alto valor: la validación de la maestría del jugador.

19. Screenplay / Gameplay - Nivel 1: El Despertar Criogénico

19.1. Propósito

Relato detallado del flujo jugable y cinematográfico del primer nivel, pensado para servir de base a storyboards y animáticas.

19.2. ![[Map of Level 1 Concept.png]]

19.3. 1. Cinemática de Inicio

- Oscuridad total. Sonido de respiración y latidos.
- Fade in: Elías abre los ojos dentro de la cápsula criogénica. Condensación en el vidrio, luces azules parpadean.
- HUD: Diagnóstico de vida, mensajes de la IA (texto flotante: “Protocolo Arca activo”).
- Elías golpea el vidrio, la cápsula se abre con vapor y ruido hidráulico.

19.4. 2. Primeros Pasos (Gameplay)

- Control al jugador. Elías sale tambaleante, cámara en tercera persona.
- Tutorial de movimiento: caminar, correr, saltar.
- El entorno: hileras de cápsulas, niebla azul, luces de emergencia.
- Primer obstáculo: plataforma caída, salto sencillo.

19.5. 3. Amenaza Silenciosa

- Dron DDC patrulla el pasillo. Luz cian barre el entorno.
- Elías debe ocultarse tras cajas o moverse en sigilo.
- Si es detectado: alarma breve, puerta se sella, debe resolver un puzzle rápido (realinear circuitos).

19.6. 4. Interacción Ambiental

- Panel de control: Elías activa energía secundaria, desbloquea una ruta.
- Fuga de plasma bloquea el paso, requiere saltar en el momento justo.
- Correa de transporte: tutorial de uso para cruzar un túnel de viento.

19.7. 5. Clímax y Cliffhanger

- Elías llega al ventanal principal. Vistas de la nebulosa exterior.
 - Cinemática: aparece Odisea como holograma etéreo.
 - Diálogo:
 - Odisea: “Todo controlado, Elías. Protocolo Arca los protege.”
 - Elías: duda, pregunta por fallos técnicos.
 - Odisea responde evasiva, glitches en la voz.
 - Repentina explosión de plasma. Elías salta, cae de rodillas, jadea.
 - Panel de alerta: “Trayectoria desviada. Destino: desconocido”.
 - Odisea susurra: “Confía en mí, Elías.”
-

19.8. Notas para Storyboard

- Enfatizar la atmósfera opresiva y la soledad.
- Contrastar la aparente calma de la IA con el deterioro ambiental.
- Usar cambios de luz y niebla para guiar la atención y el ritmo.
- Cada sección puede dividirse en viñetas para storyboard visual.

20. Storyboard Narrativo y Jugable — Nivel 1: El Despertar Criogénico

Locación: Módulo Criogenia (Popa) — Pasillo en L + Cubículo de Control **Duración estimada:** 5–8 minutos **Objetivo narrativo:** Elías despierta sin contexto. La nave parece segura. La IA parece benéfica. Nada es lo que parece. **Objetivo jugable:** Aprender movimiento, interacción con consola, lógica terminal→botón→puerta.

20.1. BEAT 1 — El Despertar (Cinemática, ~30s)

Cámara: Interior de la cápsula criogénica. Oscuridad total.

Acción: - Sonido de latidos y respiración lenta que acelera. - Fade in: condensación en el vidrio. Luces azules parpadean al otro lado. - Texto flotante en el HUD: *“Protocolo Arca activo. Bienvenido, Unidad 782-C.”* - Elías golpea el vidrio. La cápsula se abre con vapor y sonido hidráulico. - **Dolly shot scriptado:** la cámara barre lentamente hacia atrás revelando el pasillo — hileras interminables de cápsulas a ambos lados, niebla cian al nivel del suelo, luces de emergencia rojas parpadeando en el techo. La escala es aplastante.

Emoción objetivo: Desorientación. Soledad. Pequeñez ante la máquina.

20.2. BEAT 2 — Primeros Pasos (Gameplay, ~60s)

Layout: Primer tramo del pasillo en L (eje norte-sur).

[CÁPSULA ELÍAS]

holopantalla 1

holopantalla 2

holopantalla 3

[ESQUINA - CUBÍCULO →]

Acción jugable: - Control al jugador. Elías sale tambaleante (animación de inicio lenta, sin sprint disponible aún). - Las **holopantallas** en las paredes emiten luz cian intensa — muestran: lista de pasajeros (49.000 nombres), fecha de partida, estado de la misión (“EN CURSO — Año 3”), conteo de cápsulas activas. - No son interactivables. Son ambientales. El jugador que se acerca las lee; el que no, las ignora. - Al llegar a la esquina, el cubículo es visible: **la habitación más iluminada del pasillo**, contraste deliberado contra el rojo de emergencia del corredor.

Narrativa ambiental: Las holopantallas no dicen nada alarmante. Todo parece en orden. Eso es la mentira.

Emoción objetivo: Curiosidad. Asombro silencioso.

20.3. BEAT 3 — El Cubículo de Control (Gameplay + Narrativa, ~90s)

Layout: Alcoba lateral en la esquina del L, abierta al pasillo. Espacio para 3–4 personas.

[PANTALLA ALERTA]

[PANTALLA SISTEMA]

[TERMINAL] ← interactuable (naranja)

[PANTALLA MAPA]

abierto al pasillo

Acción jugable: 1. El jugador entra al cubículo. Tres pantallas grandes muestran: mapa de la nave, logs del sistema, y una **pantalla de alerta en rojo** que dice “*ACCESO RESTRINGIDO — Sector E-7 bloqueado*”. 2. La **terminal central** (naranja, parpadeante) es el único objeto interactuable. Prompt: “[E] Hackear terminal”. 3. Al pulsar E: la terminal muestra una interfaz de texto simple. Elías escribe un comando (animación automática). La pantalla de alerta cambia a verde: “*Acceso autorizado. Seguro de puerta desactivado.*” 4. Sonido: un *click* metálico lejano al fondo del segundo tramo — el botón de la puerta se activa.

Momento narrativo — Primera aparición de Odisea: - Al hackear la terminal, el holograma de Odisea aparece brevemente sobre la pantalla central. - Voz tranquila, casi maternal: “*Elías. Es tarde para estar despierto. El Protocolo Arca garantiza tu seguridad. Te recomiendo volver a la cápsula.*” - Elías no responde. El jugador decide si se queda o avanza. - Odisea no bloquea. Solo observa. Solo sugiere.

Emoción objetivo: Primer escalofrío. La IA sabe que estás ahí. Y no quiere que avances.

20.4. BEAT 4 — El Segundo Tramo (Gameplay, ~60s)

Layout: Segundo tramo del pasillo en L (eje este-oeste).

[ESQUINA]

holopantalla 4 (ahora muestra: "Trayectoria activa: DESVIADA")

holopantalla 5 (log de error: fecha - hace 6 meses)

[BOTÓN] ← naranja, ahora activo (luz verde)

[PUERTA] ← bulkhead cerrada

Acción jugable: 1. El jugador avanza por el segundo tramo. Las holopantallas aquí ya no son neutras — muestran errores del sistema, una trayectoria marcada como “DESVIADA”, un log de hace meses que nadie leyó. 2. El **botón** al lado de la puerta ahora tiene luz verde (antes estaba apagado/rojo). Prompt: “[E] Abrir puerta”. 3. Al pulsar: la puerta bulkhead se abre con sonido neumático pesado. Luz del corredor siguiente se filtra.

Detalle de diseño: - Si el jugador intenta pulsar el botón *antes* de hackear la terminal: no responde, luz roja, sin prompt. Sin mensaje de error. Simplemente no pasa nada — el jugador entiende solo que debe buscar otra forma.

Emoción objetivo: Satisfacción de la solución. El mundo respondió a una acción tuya.

20.5. BEAT 5 — Cliffhanger de Salida (Cinemática breve, ~15s)

Acción: - Elías cruza la puerta. Al pasar el umbral: pequeño **screen shake** + golpe metálico lejano. - La puerta se cierra detrás — sonido definitivo. No hay vuelta. - En el corredor siguiente, la niebla es más densa. Las luces son más tenues. - Odisea susurra (sin holograma, solo voz): “*Confía en mí, Elías.*” - Pausa. Silencio. El jugador avanza.

Emoción objetivo: Punto de no retorno. La calma antes de lo que viene.

20.6. Resumen de Interactables

Objeto	Tipo	Estado inicial	Acción	Resultado
Holopantallas (×5)	Pasivo	Siempre activas	Ninguna	Narración ambiental
Terminal del cubículo	Activo (naranja)	Disponible	[E] Hackear	Activa el botón de la puerta

Objeto	Tipo	Estado inicial	Acción	Resultado
Botón de la puerta	Activo (naranja)	Inactivo (rojo)	[E] Abrir (solo si terminal hackeada)	Abre la puerta bulkhead
Puerta bulkhead	Pasivo	Cerrada	—	Se abre al pulsar botón
Holograma de Odisea	Cinemático	Trigger al hackear	—	Primer diálogo de Odisea

20.7. Checklist de Legibilidad

- ☐ La ruta crítica nunca requiere girar $>90^\circ$ sin una señal visual (luz o partícula)
- ☐ El cubículo es el punto más brillante del pasillo — visible desde el spawn
- ☐ El botón inactivo no tiene prompt — el jugador no sabe que existe hasta que lo activa
- ☐ El holograma de Odisea aparece una sola vez — no satura el espacio
- ☐ Las holopantallas del segundo tramo revelan la anomalía *después* de que el jugador ya se comprometió a avanzar

20.8. Notas de Implementación (Godot 3)

- **Dolly shot:** Camera con Path + PathFollow en AnimationPlayer, duración 4s, luego `emit_signal("cutscene_finished")` para devolver control al PlayerController.
- **Terminal:** InteractableEntity con `interaction_text = "Hackear terminal"`, emite señal `interacted` → conectada a `_on_terminal_hacked()` en el nodo de la sala.
- **Botón:** InteractableEntity con `requirements` bloqueado por defecto. `_on_terminal_hacked()` llama `boton.unlock()` que activa el prompt y cambia material de rojo a naranja/verde.
- **Holograma Odisea:** AnimationPlayer + AudioStreamPlayer disparado por `_on_terminal_hacked()` con delay de 0.5s.
- **Screen shake al salir:** Camera.offset con Tween de 0.3s, amplitud 0.05.

21. The Architecture of Precision and Perspective: A Socio-Technical Analysis of Platforming Kinesthetics and Third-Person Spatial Mechanics

The optimization of player engagement in modern interactive media necessitates a rigorous synthesis of kinetic physics, cognitive psychology, and spatial engineering. At the intersection of these disciplines lies the “science of fun,” a quantifiable framework that governs how movement, perspective, and challenge coalesce to create a state of optimal experience. Precision platformers and third-person action games represent the polarities of this design spectrum—one focusing on the micro-seconds of input fidelity within a two-dimensional plane, and the other on the macro-management of spatial awareness and cinematic immersion within a three-dimensional volume.

The relationship between the player avatar and the digital environment is characterized by a continuous feedback loop where the game takes output and introduces it back as input.[1] In precision genres, the “fun” is not merely the absence of frustration but the achievement of mastery through a calibrated cycle of failure and triumph. This report deconstructs the metrics and methodologies that define these genres, examining the underlying mechanisms of flow, the technical nuances of forgiveness systems, and the rational design of level geometry.

21.1. The Kinesthetics of Movement and Response

The fundamental unit of engagement in any character-driven game is the movement mechanic, or what is termed game feel. This sensation is the result of real-time control over virtual objects in a simulated space, emphasized through polish and instantaneous feedback.[2] To achieve a state of high-fidelity control, designers must bridge the gap between the player’s intention and the on-screen execution, a process that relies heavily on the manipulation of acceleration, friction, and gravity curves.

21.2. Animation Curves and the Metric of Snappiness

The transition from a stationary state to maximum velocity is a critical metric in defining the responsiveness of a character. Industry standards suggest that for precision platformers, reaching top speed should be nearly instantaneous to ensure the controls feel snappy rather than floaty.[3] Snappiness is a functional requirement for high-difficulty titles because it minimizes the unpredictable “ramp-up” period where a player cannot accurately judge the character’s final position. When a character reaches top speed almost instantly, the player gains a heightened sense of causality.[3, 4]

Professional implementations often utilize Scriptable Objects to store movement presets, which include discrete AnimationCurves for different states.[3] These curves govern the acceleration and deceleration of the character when on the ground and in the air. By separating these curves, designers can allow for high friction on the ground to enable precise stopping, while maintaining lower friction in the air to allow for momentum conservation.[3, 5] A common approach involves four distinct curves:

Curve Designation	State	Functional Purpose
Ground Acceleration	Grounded	Defines how quickly the character reaches the walk/run cycle. [3]
Ground Deceleration	Grounded	Determines the “slide” distance when input is released. [3]
Air Acceleration	Aerial	Governs the player’s ability to adjust trajectory mid-jump. [3]
Air Deceleration	Aerial	Controls the retention of horizontal momentum in flight. [3]

These curves are often supplemented by a boolean check for momentum reset. Choosing whether to reset momentum when turning 180 degrees is a vital design decision; resetting it immediately feels more responsive but less realistic, whereas allowing a turning arc adds “weight” but reduces precision.[3] For precision games like *Super Meat Boy* or *Celeste*, the former is almost universally preferred.[6]

The Physics of the Vertical Leap

In precision platformers, the jump is rarely a simple parabolic arc governed by Newtonian physics. Real physics often look “wrong” in a digital context; for instance, a human jumping in real life only reaches about half their height and stays in the air for a second.[7] In a game, this would result in a jump of approximately 16 pixels, which is often insufficient for level design needs. Instead, designers manipulate velocity and gravity frames to create a more expressive and controllable arc.

A typical jumping implementation involves giving the object a large negative Y velocity (e.g., -6 pixels per frame) and then applying a smaller constant gravity (e.g., 0.3 pixels per frame) to bring the object back down.[7] However, the most sophisticated systems use variable gravity. In titles such as *Celeste* or *Super Meat Boy*, the game handles a jump normally while the button is held, but releasing the button acting as a brake, cutting the jump short by increasing gravity or immediately zeroing the upward velocity.[6] This is known as “jump braking” and is essential for precise vertical navigation.

Celeste further refines this by applying “halved gravity” at the peak of a jump if the button is held.[8] This subtle manipulation extends the “hang time” at the apex, providing the player with a larger window to adjust for a landing or to plan their next move. This is a classic example of “polish”—a mechanic that is difficult for the player to see consciously but contributes significantly to a pleasant game feel.[2, 8]

Forgiveness Systems and the “Kindness” of Difficulty

High-difficulty games often employ “invisible” systems to widen the success window for players, ensuring that deaths feel like the result of personal error rather than mechanical rigidity. These

systems are essential for maintaining the player’s psychological state of flow by reducing frustration caused by “near-miss” failures. The goal is to make the game “kind” even when it is difficult.[8]

Coyote Time: Bridging Perception and Action

Coyote Time refers to a brief period—typically 5 to 10 frames—where the player can still jump after falling off a ledge.[8, 9, 10] This mechanic addresses the latency between the player perceiving they are at the edge and the physical press of the button. Effectively, this provides a few frames for the player to jump in the air, improving the curve of an ideal jump from a slope with an immediate drop to a more helpful leeway on both sides of the edge.[10]

In *Celeste*, Coyote Time is set to 5 frames.[11] This window allows for advanced maneuvers, such as obtaining the maximum possible distance from an “extended hyper” jump or increasing the height of a “dream jump” when exiting blocks.[11] Without this forgiveness, players would often feel they were “robbed” of a jump that they perceived as occurring while still on the platform.

Jump Buffering: Caching Player Intention

While Coyote Time handles late inputs, Jump Buffering handles early inputs. If a player presses the jump button a short time before landing on the ground, the game caches this input and executes the jump on the exact frame the character becomes grounded.[9, 12] This prevents the “dead input” problem that occurs when a player attempts to chain jumps but is off by a fraction of a second.

Standard implementations use a timer—often with a duration of 0.1 seconds—that starts when the jump button is pressed in mid-air.[12, 13] If the character hits the floor while this timer is active, the jump method is called immediately. Designers must ensure this timer is not too long, as it could lead to “pogo-sticking” where the character jumps repeatedly without the player intending to do so.[12]

Corner and Dash Correction

Advanced error correction extends beyond timing and into spatial positioning. Corner Correction is a sophisticated mechanic where the game “wiggles” or “pops” the character around obstacles if they clip a corner by a small margin.[8]

In *Celeste*, this is applied to both jumps and dashes. If a player dashes sideways and clips a corner, the game pops them up onto the ledge.[8] Similarly, if they bonk their head on a corner during a jump, the game nudges them to the side. These corrections are often measured in pixels:

- **Jump Corner Correction:** Wiggles the character to the side if they hit a corner.[8]
- **Dash Corner Correction:** Pops the character onto a ledge if they clip it during a horizontal dash.[8]
- **Wall Jump Window:** Allows a wall jump even if the player is 2 pixels away from the wall.[8]
- **Super Wall Jump Window:** Extends the window to 5 pixels (more than half a tile) for more demanding maneuvers.[8]

These pixel-level adjustments ensure that the character’s hitbox feels “slimmer” for the player’s benefit, effectively reducing the frequency of frustrating collisions that would otherwise break the game’s flow.

The Psychology of Flow, Fiero, and Engagement

The “fun” of precision platforming and high-intensity combat is rooted in the psychological state of flow. Proposed by Mihaly Csikszentmihalyi, flow is the optimal state of consciousness where an individual is fully immersed in an activity, characterized by a balance between challenges and skills.[14, 15, 16] When a player’s skill matches the difficulty of the task, they enter the “flow channel,” avoiding both the boredom of tasks that are too easy and the anxiety of those that are too hard.[15]

The Nine Elements of Flow in Gaming

Flow in video games is sustained by nine common elements, which can be categorized into antecedent conditions and internal experiences [14, 15]:

1. **Clear Goals:** The player understands exactly what success looks like in the current task (e.g., reaching the next checkpoint).[15]
2. **Immediate Feedback:** Direct and timely feedback helps players adjust their actions and stay engaged.[15]
3. **Balance Between Challenge and Skill:** The difficulty curve matches the player’s progression.[14, 15]
4. **Merging of Action and Awareness:** The player and the game unit (character) feel as one.[14]
5. **Exclusion of Distractions:** The high level of concentration required removes external reality from consciousness.[14, 17]
6. **No Worry of Failure:** A low punishment for death (quick respawns) encourages experimentation and reduces stress.[14]
7. **Loss of Self-Consciousness:** The player is no longer aware of themselves as separate from the activity.[15]
8. **Altered Sense of Time:** Time may seem to speed up or slow down.[15, 17]
9. **Autotelic Experience:** The activity is rewarding for its own sake rather than for external rewards.[15, 17]

Fiero: The Emotional Reward of Triumph

While flow is the state of deep concentration, Fiero is the specific emotion of triumph and pride experienced upon accomplishing a difficult task.[14] Derived from the Italian word for pride, Fiero is often expressed through shouting, throwing up hands, or an elevated heart rate.[14] Highly successful games are those that require high levels of intensity and concentration to achieve this state.

Fiero is a primal chemical reward that motivates players to return to play, even after experiencing frustration.[14] Baumann, Lurig, and Engeser suggest that flow is a loop toward achieving Fiero, asserting that players arrive at a state of pride only after completing a part of the game that requires significant upfront frustration.[14] This explains why games like *Super Meat Boy* or *Dark Souls* are “fun” despite their extreme difficulty; the depth of the frustration makes the eventual Fiero more intense.

Rational Level Design (RLD) and the Metrics of Difficulty

To consistently deliver the state of flow, designers utilize Rational Level Design (RLD), a framework that quantifies difficulty through metrics rather than intuition.[18] RLD allows designers to fine-tune game elements to ensure the player remains within the flow channel as their familiarity and skill increase.[18]

The Four Primary Metrics of Difficulty

The Rational Design Handbook outlines four primary stages of fine-tuning that impact the overall difficulty of a set of level metrics [18]:

1. **Situational Awareness:** This is the most powerful metric for altering difficulty. it controls the amount of information available to the player across three timeframes: second-to-second (immediate puzzles/combat), minute-to-minute (local geography/minimap), and hour-to-hour (overarching goals).[18]
2. **Noise:** Superfluous information used to overwhelm the player and obscure the simplicity of a task. This impacts player comprehension and can be visual, auditory, or mechanical.[18]
3. **Time Pressure:** Increasing difficulty by reducing the amount of time a player has to act on information. This is placed after situational awareness because a player cannot act on a problem until they are aware of it.[18]
4. **Symmetry:** Expressed as a ratio of possibilities between the player and the environment or enemy. A 1:1 ratio is symmetry (equal options), while moving away from this creates asymmetry to fine-tune difficulty based on progression.[18]

RLD Metric	Typical Scaling / Ratio	Impact on Difficulty
Situational Awareness	+/- 20% (0.8 to 1.2) [18]	Highest: Alters the fundamental perception of the challenge.
Noise	Superfluous UI/Particles	Medium: Increases cognitive load and distraction.
Time Pressure	Seconds available to act	High: Forces faster processing and higher error rates.
Symmetry	1:1 (Symmetric) to N:1	Medium: Adjusts the “fairness” of the interaction.

Designing with these metrics involves establishing a baseline “Perceptual Value of 1.0” during testing.[18] Designers then refactor difficulty by applying deviation ratios. For example, if situational awareness is increased to 1.3, the corresponding puzzle or combat values are multiplied by that factor. Any value exceeding a ceiling (often 8.0) is considered too difficult for the intended audience and is removed.[18]

Spatial Metrics and Environmental Scaling

In level design, metrics are used to define the physical dimensions of the world to ensure they complement the player’s movement parameters. For 3D third-person games, a commonly acknowledged rule is that the environment should be scaled up by approximately 33%.[19] This additional space is necessary for several reasons:

- **Camera Clearance:** It allows space for the third-person camera to move through the environment without clipping into walls or obscuring the player’s view.[19]
- **Maneuverability:** It provides large enough arenas and wide enough corridors (roughly 2.7 to 3.6 meters) so that players don’t “cling” to walls or fall off bridges accidentally.[20, 21]
- **Combat Distance:** Map sizes are often scaled to match the range of the character’s abilities. For instance, close-combat maps might be 800x800m, while long-range maps extend to 1200x1200m.[20]

Environmental Metric	Typical Dimension	Rationale
Room/Hallway Width	2.7m - 3.6m [21]	Allows for movement without wall-clipping.
Main Path Width	~5.0m [21]	Clearly distinguishes the primary route.
Side Path Width	~3.0m [21]	Signals a secondary or secret path.
Cover Distance	2.0m - 3.0m [21]	Balances safety with vulnerability during movement.

Visual Language, Affordances, and Signposting

To maintain flow, a level must be “readable.” Players should intuitively understand what they can interact with and where they should go without overt instructions.[22, 23] This is achieved through the strategic use of affordances and signifiers.

Affordances and Signifiers

Affordances are properties of an object that suggest its use (e.g., a ladder suggests climbing, a door suggests opening).[22, 23] Signifiers are visual or audio cues that guide the player’s attention to these affordances. For instance, a shimmering light on a ladder or a yellow cloth on a ledge serves as a signifier that it is interactable.[22]

Effective signposting prevents frustration by ensuring design intentions are clearly communicated.[23] Techniques include:

- **Lighting:** Placing a bright light source at the destination of a dark environment to create a natural “leading line”.[22, 24]
- **Color Coding:** Using consistent colors (e.g., yellow for climbable surfaces, red for explosive barrels) to establish a visual vocabulary.[22, 23]
- **Landmarks:** Large, unique structures that serve as a point of reference for navigation and orientation.[21]

The Four Pillars of Visual Language

Level designers utilize four primary pillars of visual language to guide behavior subconsciously [25]:

1. **Shape Language:** Using forms to convey meaning. Narrow pathways suggest linear progression, while open spaces imply exploration or freedom.[25]
2. **Environmental Storytelling:** Using contextual details (e.g., footprints leading to a cave, a battlefield littered with weapons) to guide gameplay without cutscenes.[25]
3. **Scripted Scenes:** Constraining camera movement or funneling players through a bottleneck to ensure they see a specific dramatic or educational event.[25]
4. **Symbol Language:** Using icons, murals, or markers (e.g., a glowing handle for positive interaction, debris for negative interaction) to communicate directly with the player.[25]

Third-Person Camera Mechanics and the Illusion of Depth

The transition from 2D to 3D introduces significant challenges in perspective and depth perception. Perspective view in computer graphics mimics real-world vision by making objects smaller as they move away and causing parallel lines to converge at vanishing points.[26] However, this “foreshortening” can make it difficult for players to judge jump distances or the positions of enemies in a 360-degree space.[27, 28]

The “One-Shot” Camera of God of War (2018)

In *God of War (2018)*, the developers opted for an “intimate” over-the-shoulder camera to highlight engine features like high-poly characters and advanced shaders.[29] This choice influenced the entire design, leading to close combat distances and a more personal connection with Kratos.[29, 30] However, this restrictive field of view (FOV) created a trade-off: players often struggle to see enemies behind them, necessitating “hack-ey” tricks like a quick-turn button and HUD indicators that warn of off-screen threats.[31, 32]

Camera Parameter	Default Value (GoW)	Impact on Gameplay
Field of View (FOV)	90 Degrees [33]	Restricts peripheral vision; enhances cinematic feel.
Offset (X, Y)	Shoulder Position	Creates an “intimate” view but can obscure pathfinding. [34]
Distance	“Intimate” / Close	Emphasizes combat impact but reduces scale. [32]

While some players find the default FOV claustrophobic, the development team used this proximity to manage focus. By pulling the camera back or expanding the FOV (as seen in PC mods), players might see characters or effects “teleporting” into a scene, breaking the “one-shot” illusion that the developers maintained through scripted moments.[31, 35, 36]

Enhancing Depth Perception in Virtual Spaces

To assist players in 3D navigation, designers use subtle depth cues. One such technique is the “Landing Shadow” or “Blob Shadow.” By projecting a simple, dark circle directly beneath the player’s character in mid-air, the game provides a visual anchor that allows the player to triangulate their exact position over a platform, regardless of the camera’s angle.[37]

Other cues include:

- **Atmospheric Perspective:** Using fog or color gradients that vary with distance to enhance pre-attentive depth perception.[28]
- **Texture Blur:** Increasing blurriness with distance to resemble natural depth cues and maintain visual integrity while covertly enhancing depth estimation.[28]
- **Size Scaling:** Using repeated assets (like palms in racing games) at different sizes to create the illusion of depth.[38]

The Failure Cycle: Death and Rebirth Metrics

In precision genres, failure is not a terminal state but a central component of the experience. The design of what happens when a player dies—the “death and rebirth taxonomy”—directly impacts mastery, challenge, and immersion.[39, 40]

The Taxonomy of Platformer Death

A study of 62 platformer games identified five notable dimensions of death and rebirth [39, 40]:

1. **Obstacles:** The cause of death (e.g., pits, enemies, spikes).
2. **Death Conditions:** The trigger (e.g., health reaching zero, falling off the screen).
3. **Aesthetics:** The visual and audio representation (e.g., a “glitch” sound, character fading out).[41]
4. **Changes to Player Progress:** What is lost (e.g., items, hearts, progress toward a checkpoint).[40, 41]
5. **Respawn Locations:** Where the cycle restarts (e.g., start of game, start of level, last checkpoint, or manual save point).[39, 41]

The Metric of Respawn Time

The duration of the death-to-respawn transition is a critical variable. In precision platformers like *Super Meat Boy* or *Celeste*, mistakes are easy and frequent, so the game uses instant resets (1-2 seconds) to keep the player focused and motivated.[42] These short breaks reduce the punishment of death, allowing for “repetition-based learning”.[42, 43]

Contrast this with “slower” games like those in the *Souls* series, where death animations and loading screens can take 10 to 15 seconds.[42] This downtime serves as a “wave’s ebb,” providing a controlled burst of intensity followed by a period where the player must analyze their failure before retrying. In high-intensity sessions, if the intensity is always high, it’s never high; downtime is a valuable tool for pacing.[42]

Game Genre	Challenge Duration	Respawn Time	Purpose
Precision Platformer	~20 Seconds [42]	1-2 Seconds [42]	Maintain high focus/flow.
Hard Action (Souls-like)	Minutes	10-15 Seconds [42]	Encourage analysis/caution.
Multiplayer Raid (Destiny)	Long Encounters	40+ Seconds [43]	Difficulty check; force team wipe.

In multiplayer contexts like *Destiny*, long respawn timers (40+ seconds) in platforming sections are often criticized as “overboard” but are intended to increase the consequence of failure, making a full team wipe more likely if multiple players fail.[43] However, for practice-based learning, many argue that quick attempts are superior for building skill.[43]

Technical Synthesis: Analyzing Celeste’s Movement Metrics

The movement in *Celeste* is a benchmark for the genre because its metrics are transparent yet deeply layered. The game uses a 320x180 resolution, meaning even small pixel offsets are significant.[8] Its movement can be categorized into discrete “techs” that utilize specific speed values [5]:

- **Horizontal Movement:** Walking/mid-air movement caps at 90. Jumping adds 40 speed horizontally and sets vertical speed to 105.[44]
- **Dashing:** Sets speed to 240 for 10 frames, then resets to 160.[44]
- **Hyperdash:** Dashing diagonally into the ground and jumping grants 325 speed but only half the normal jump height.[5]

- **Wavedash:** A hyperdash from mid-air that hits the ground and jumps, allowing the player to keep the 325 speed and gain a full jump height if timed correctly.[5]

These mechanics rely on precise collision detection. For example, to grab a wall, Madeline must be on one of the two pixels closest to the wall and not be moving upward.[5] A “cornerboost” can be performed by climb-jumping off the top 5-7 pixels of a wall, which allows the player to maintain horizontal momentum.[5, 11]

Momentum Storage and Refunding

Celeste uses “Stamina Refunds” to resolve input ambiguity.[8] If a player jumps straight up while grabbing a wall (which consumes high stamina) but then immediately presses away from the wall, the game “refunds” the stamina and belatedly converts the action into a horizontal wall jump. This grace window ensures that the game interprets the player’s true intention rather than punishing a minor input sequencing error.[8] This is the ultimate “science of fun”: using complex underlying logic to create an experience that feels simple, responsive, and fair.

Conclusion: The Integrated Science of Engagement

The “fun” of precision platformers and third-person mechanics is not an accident but the result of a deliberate, metric-driven engineering process. By quantifying movement through Animation-Curves, difficulty through the RLD framework, and spatial awareness through environmental scaling, designers can create experiences that consistently induce the state of flow.

The most successful games are those that balance high challenge with extreme leniency. Systems like *Coyote Time* and *Corner Correction* widen the success window at the micro-level, while instant respawns and clear visual signposting maintain the momentum of the experience at the macro-level. Whether it is the 2-pixel margin of a wall jump in *Celeste* or the 33% scaling of a hallway in a third-person shooter, these metrics are the “invisible hands” that guide players toward Fiero—the ultimate reward of the gaming experience. Professional design in these domains must continue to iterate on these quantitative benchmarks, ensuring that the “science of fun” remains grounded in the human perceptual and psychological limits of the player.

-
1. Game systems: Feedback loops and how they help craft player experiences, <https://machinations.io/article/systems-feedback-loops-and-how-they-help-craft-player-experiences>
 2. Analysis and Generation of Flow in 3D Jump’n’Run Games, <https://downloads.hci.informatik.uni-wuerzburg.de/2024-CoG-JumpAndRunFlow.pdf>
 3. InDepth: Movement in Unity using AnimationCurves | by Gustav Corpas | Dev Genius, <https://blog.devgenius.io/indepth-movement-in-unity-using-animationcurves-1dba668dc777>
 4. Mechanics of Game Feel, <http://dtc-wsuv.org/wp/dtc338-engines/files/2017/01/Mechanics-and-Metrics-of-Game-Feel-Steve-Swink.pdf>
 5. Tech - Celeste Wiki, <https://celeste.ink/wiki/Tech>
 6. Platformer Game Design (Definition, Fundamentals, Mechanics), <https://gamedesignskills.com/game-design/platformer/>
 7. Calculating jump speed - NESDev Forum, <https://forums.nesdev.org/viewtopic.php?t=13765>
 8. Celeste & Forgiveness - Maddy Makes Games, https://www.maddymakesgames.com/articles/celeste_and_forgiveness/

9. Coyote time and Jump Buffer for platformer games - Developer Forum | Roblox, <https://devforum.roblox.com/t/coyote-time-and-jump-buffer-for-platformer-games/2809273>
10. Building Coyote Time in a 2D Platformer - World of Zero, <https://worldofzero.com/videos/building-coyote-time-in-a-2d-platformer/>
11. Moves | Celeste Wiki - Fandom, <https://celestegame.fandom.com/wiki/Moves>
12. Improve Your Game Feel With Coyote Time and Jump Buffering - YouTube, https://www.youtube.com/watch?v=97_jvSPoRDo
13. Coyote Time & Jump Buffering In Unity - YouTube, https://www.youtube.com/watch?v=RFix_Kg2Di0
14. Fiero and Flow in Online Competitive Gaming: - IGI Global, <https://www.igi-global.com/viewtitle.aspx?TitleId=315493&isxn=9781668475898>
15. Design components of serious game based on flow theories - Clausius Scientific Press, https://clausiuspress.com/assets/default/article/2025/05/31/article__1748743350.pdf
16. (PDF) Toward an understanding of flow in video games - ResearchGate, <https://www.researchgate.net/publication/315493>
17. Flow and Immersion in Video Games: The Aftermath of a Conceptual Challenge - Frontiers, <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2018.01682/full>
18. The Rational Design Handbook: Four Primary Metrics, <https://www.gamedeveloper.com/design/the-rational-design-handbook-four-primary-metrics>
19. A Rational Approach To Racing Game Track Design - Game Developer, <https://www.gamedeveloper.com/design/a-rational-approach-to-racing-game-track-design>
20. Featured Blog | Designing maps that complement game mechanics, <https://www.gamedeveloper.com/design/maps-that-complement-game-mechanics>
21. Level Design, <https://memoof.me/download/445/pdf/445.pdf>
22. Mastering the Invisible : How Affordances & Signifiers Shape Player Experience in Level Design | by Genesis | Medium, https://medium.com/@Genesis_Design/mastering-the-invisible-how-affordances-signifiers-shape-player-experience-in-level-design-64082c602fa0
23. Level Design: Understanding a level - Game Developer, <https://www.gamedeveloper.com/design/level-design-understanding-a-level>
24. The Influence of Intrinsic Cues on Navigational Choices in Virtual Environments. - University of Huddersfield Repository, <https://eprints.hud.ac.uk/id/eprint/34448/1/FINAL%20THESIS%20-%20Marples.pdf>
25. How to Use Visual Language for Intuitive Level Design | UXPin, <https://www.uxpin.com/studio/blog/how-to-use-visual-language-for-intuitive-level-design/>
26. What is Perspective View? How Does it Work? | Lenovo UK, <https://www.lenovo.com/gb/outletgb/en/gb/view/>
27. DepthScape: Authoring 2.5D Designs via Depth Estimation, Semantic Understanding, and Geometry Extraction - arXiv, <https://arxiv.org/html/2512.02263v1>
28. Subtle Cueing For Improving Depth Perception in Virtual Reality - IEEE Xplore, <https://ieeexplore.ieee.org/iel8/10765357/10765151/10765485.pdf>
29. Level Design in a Day: Your Questions, Answered - Game Developer, <https://www.gamedeveloper.com/design/level-design-in-a-day-your-questions-answered>

30. Pattern Language For Game Design | PDF | Narrative - Scribd, <https://www.scribd.com/document/71245Language-for-Game-Design>
31. God of War (2018) restrictive FOV is such a turn off. : r/GodofWar - Reddit, https://www.reddit.com/r/GodofWar/comments/196yw3e/god_of_war_2018_restrictive_fov_is_such
32. God of War Camera Angle—Thoughts? : r/GodofWar - Reddit, <https://www.reddit.com/r/GodofWar/com>
33. Tutorial: How to fix God of war 2018 Camera Fov - Pc version - YouTube, <https://www.youtube.com/watch?v=XjSIm7zpfz4>
34. Xbox – Out Of Lives, <https://www.outoflives.net/category/gaming/xbox/feed/>
35. PSA: You can easily add an FOV slider to God of War PC using ‘Flawless Widescreen’ : r/GodofWar - Reddit, https://www.reddit.com/r/GodofWar/comments/sb1j4v/psa_you_can_easily_ac
36. FOV (Field Of View) Mod for God of War : r/GodofWar - Reddit, <https://www.reddit.com/r/GodofWar/>
37. Dribbble Favorites - Ivo’s Design Inspiration Gallery, <https://ivomynttinen.com/gallery/dribbble-favorites/>
38. 2.5D - Wikipedia, <https://en.wikipedia.org/wiki/2.5D>
39. Death & Rebirth in Platformer Games - ALT Games Lab, <https://altgameslab.soe.ucsc.edu/death-and-rebirth-in-platformer-games/>
40. (PDF) Death and Rebirth in Platformer Games - ResearchGate, <https://www.researchgate.net/publication>
41. (PDF) Die-r Consequences: Player Experience and the Design of Failure through Respawn-ing Mechanics - ResearchGate, https://www.researchgate.net/publication/354020467_Die-r_Consequences_Player_Experience_and_the_Design_of_Failure_through_Respawning_Mechanics
42. How long should death take and why? : r/gamedesign - Reddit, <https://www.reddit.com/r/gamedesign/co>
43. Respawn timers of 40+ seconds, especially in non-combat areas, is wholly ridiculous and needs to stop. : r/DestinyTheGame - Reddit, <https://www.reddit.com/r/DestinyTheGame/comments/10>
44. Asking about the mechanics : r/celestegame - Reddit, <https://www.reddit.com/r/celestegame/comments/>

Part IV.

Anexos - Canon

22. Technical Specification: Deterministic Interactable System (Odisea V2)

22.1. Objective

Create a base framework for objects that toggle states (Open/Closed, On/Off, Active/Inactive) through player interaction.

The system must ensure 100% mathematical determinism for replay integrity, inspired by the state-logic of Cogito but optimized for Odisea's snapshot architecture.

22.2. Core Architecture

22.2.1. InteractableBaseV2.gd (Abstract Class)

Inherits from `KinematicBody` (preferred for floor velocity inheritance) or `Spatial`.

22.2.2. State Variables

- `is_active`: bool: The logical state (`true` = open/on, `false` = closed/off).
- `anim_progress`: float: Normalized value (0.0 to 1.0) representing the current visual position.
- `target_progress`: float: The goal state (usually 1.0 or 0.0).
- `anim_speed`: float: Progress increment per second (`1.0 / duration`).

22.2.3. Core Methods

- `interact()`: Toggles `is_active` and sets `target_progress`.
- `step(dt)`: Calculated during the fixed physics step. Updates `anim_progress` towards `target_progress` and calls `_update_visuals()`.

22.3. Implementation Types

22.3.1. Sliding Door / Drawers (SlidingObjectV2)

Similar to `Cogito_Door.gd` but using relative translation.

- Logic: `translation = start_pos.linear_interpolate(start_pos + slide_vector, _ease(anim_progress))`.
- Generalization: Drawers are sliding objects with a restricted axis and specific sound triggers.

22.3.2. Rotational Levers / Valves (RotatingObjectV2)

- Logic: Applies `lerp` to a specific axis of rotation.
- Reference: Mimics `Cogito_Switch.gd` mechanics for flipping states via angular displacement.

22.3.3. Push Buttons / Consoles (ButtonV2)

- Logic: Combines a short Z-axis displacement with material emission changes.
- Feedback: `material.emission_energy = lerp(min_e, max_e, anim_progress)`.

22.4. Replay and Determinism (The Odisea Contract)

Unlike Cogito's reliance on standard `AnimationPlayers` (which can drift if not synced to physics), Odisea objects calculate their state based on a `time_accumulator` or a fixed-step increment.

```
func get_snapshot() -> Dictionary:
    return {
        "active": is_active,
        "progress": anim_progress,
        "target": target_progress
    }
```

23. Spec: Unified Interaction Framework (UIF) - Sensor-Based - Odisea Core_v2

23.1. Architectural Concept

The interaction system shifts from a precise RayCast (“pixel hunting”) to a volume-based detection (sensor). The player projects an invisible volume (box/cone) forward. Any `InteractableEntity` within this volume becomes a candidate.

The system evaluates candidates each frame to determine the best focus target based on proximity and angle. This unifies interaction logic for static objects (terminals) and dynamic objects (moving platforms, pushable boxes), ensuring all gameplay elements respect the deterministic `step(dt)` cycle.

23.2. Component Architecture

23.2.1. The Sensor (`InteractionSensor.gd`)

- Location: `core_v2/Components/Player/InteractionSensor.gd`
- Node: `Area` (child of `PlayerControllerV2`)
- Responsibilities:
 - Candidate tracking: maintain a list of overlapping interactable objects.
 - Evaluation loop (`step`): in every `step(dt)`, calculate a score for each candidate.
 - Angle: dot product with camera forward vector (center is better).
 - Distance: closer is better.
 - Line of sight: internal RayCast check to prevent interaction through walls.
 - Focus management: emit signals when the best candidate changes.

23.2.2. The Interactable (`InteractableEntity.gd`)

- Location: `core_v2/Components/Shared/InteractableEntity.gd`
- Inheritance: base class for all interactive objects.
- Core data:
 - `interaction_text`: `String` ("Open", "Push", "Activate").
 - `interaction_point`: `Position3D` (optional center for LoS checks).
 - `requirements`: `AttributeResource` (e.g. "Strength > 5").

23.3. Integration with Existing Features (“The Zoo”)

The goal is to refactor current prototypes (`MovingPlatform`, `Conveyor`, `Drawer`) into this unified system.

23.3.1. Moving Platforms (MovingPlatformV2)

- Current status: deterministic `KinematicBody` with `time_accumulator`.
- UIF integration: add an `InteractableEntity` child node if the platform has a control panel (e.g. an elevator button). The button is the interactable, not the platform itself.

23.3.2. Conveyor Belts (Conveyor)

- Current status: physics area applying force.
- UIF integration: generally passive. However, a conveyor switch object would be an `InteractableEntity` that toggles `Conveyor.active` deterministically.

23.3.3. Pushable Boxes (PushableBoxV2)

- Current status: hybrid rigid/kinematic body.

24. OdysseyScript (OYS): Input and Testing DSL Specification

24.1. Overview

OdysseyScript is a Domain-Specific Language (DSL) designed to write deterministic player input sequences within the Odisea engine. It solves the issue of fragile binary replays by allowing developers to define intent (e.g., “walk for 5 seconds”) instead of raw binary input buffers.

24.2. Structural Elements

24.2.1. Sections (SECTION ... END)

Sections group logical commands and define a success or failure context.

- Isolation: identifies exactly which part of a test failed.
- Asserts: validate state upon completion of each section.
- Reporting: the test engine logs SECTION PASSED or SECTION FAILED.

24.2.2. Sequentiality and Blocking

By default, commands are blocking. A line does not execute until the previous one completes its duration or reaches its goal.

24.2.3. The AT Modifier (Parallelism)

The AT modifier allows triggering an action while a movement command is still active.

- Syntax: [COMMAND] [VALUE] AT [TIME/DIST] [ACTION]
- Example: FW 5 AT 2 JUMP (move forward 5 units, but at second 2, press jump).

24.3. Command Vocabulary

Command	Arguments	Example	Notes
SET	prop val	SET pos (0,0,0)	Forces player state (teleport).
FW / BW	value	FW 5.0	Move forward/backward for X seconds or meters.

Command	Arguments	Example	Notes
LT / RT	degrees	LT 90	Rotate camera left/right.
JUMP	[time]	JUMP 0.5	Jump. Optionally holds the button down.
WAIT	time	WAIT 1.0	Character remains still (idle input).
LOOK	pitch	LOOK -45	Sets the vertical camera angle.
INTERACT	-	INTERACT	Presses the interaction key (E).
ASSERT	cond [msg]	ASSERT pos.y > 2	Verifies a physics or state condition.

24.4. Script Example: Backflip and Precision Test

```
// Initial Setup
SET pos (0, 0, 0)
SET rot 0

SECTION "Backflip Validation"
    FW 4.0                // Walk 4 seconds forward
    LOOK -20              // Look slightly down
    WAIT 0.5              // Small pause
    BW 2.0 AT 0.1 JUMP 0.3 // Start retreating and jump almost simultaneously
    LT 180                // Turn 180 degrees upon landing
    ASSERT pos.z < -1.0    // Verify the backflip moved us backward
END

SECTION "Object Interaction"
    FW 2.0 AT 1.5 INTERACT // Walk toward an object and interact halfway
    ASSERT door_open == true "The door did not open"
END
```

25. Uso de OdysseyScript y Replays en Odisea (2026)

Odisea ahora soporta replays y pruebas de determinismo tanto desde scripts OdysseyScript (.oys) como desde buffers JSON (.json).

25.1. 1. Escribe tu script OYS

Guarda tu secuencia en `core_v2/tests/mi_test.oys`:

```
// Test de salto y avance
SET pos (0, 0, 0)
SET rot 0
SECTION "Salto y avance"
    FW 1.0
    JUMP 0.2 AT 0.5 FW 0.5
    WAIT 0.3
    ASSERT pos.z > 1.5 "No avanzó lo suficiente"
END
```

25.2. 2. (Opcional) Genera el JSON

```
godot3-bin --no-window --script ./core_v2/utils/OYSRunner.gd ./core_v2/tests/mi_test.oys
```

Esto genera `mi_test.json`.

25.3. 3. Pruebas de determinismo (¡automáticas para ambos formatos!)

```
./runtest.sh -a ./core_v2/tests/test_determinism_v2.gd
```

El sistema detecta y ejecuta todos los .oys y .json en `core_v2/tests/` y subcarpetas.

- Si el archivo es .oys, se convierte en vivo y se usa la escena `TestScene_v2.tscn` por defecto (o la que declares con `LEVEL ...`).
- Si es .json, se usa directamente como buffer de inputs.
- Ambos tipos de archivo validan asserts y drift.

25.4. 4. Reproducir un replay en vivo

Puedes lanzar el juego con:

```
godot3-bin --main-pack project.pck --replay res://core_v2/tests/mi_test.oys
```

O con un .json:

```
godot3-bin --main-pack project.pck --replay res://core_v2/tests/mi_test.json
```

25.5. 5. Buenas prácticas

- Usa comentarios y SECTION para claridad.
- Usa asserts para validar resultados esperados.
- Si tu OYS no declara LEVEL, se usará TestScene_v2.tscn.
- Puedes mezclar archivos .oys y .json en tus tests.

25.6. Resumen rápido

Acción	Comando/Flujo
Escribir test OYS	Edita .oys en core_v2/tests/
Generar JSON (opcional)	godot3-bin --no-window --script ./core_v2/utils/OYSRunner.gd archivo.oys
Ejecutar todos los tests	./runtest.sh -a ./core_v2/tests/test_determinism_v2.gd

26. OdysseyScript (OYS) — Guía de Uso

OdysseyScript (OYS) es un lenguaje de alto nivel diseñado para describir secuencias de acciones de jugador en Odisea, permitiendo generar replays deterministas y pruebas automatizadas.

26.1. ¿Para qué sirve?

- Crear replays reproducibles para testeo y debugging.
- Escribir pruebas de determinismo para el motor y agentes.
- Documentar y automatizar secuencias de juego.

26.2. Estructura básica de un script OYS

```
// Comentario: Descripción del test
SET pos (0, 0, 0)      // Posición inicial
SET rot 0              // Rotación inicial

SECTION "Nombre de la sección"
    FW 2.0              // Avanzar 2 segundos
    WAIT 0.2           // Pausa breve
    JUMP 0.3            // Saltar, mantener 0.3s
    FW 1.0              // Avanzar 1 segundo
    ASSERT pos.z > 2 "El jugador no avanzó suficiente"
END
```

26.3. Comandos principales

- FW <valor>: Avanza. Valor puede ser segundos (ej: 2.0s) o metros (ej: 5m). Por defecto segundos.
- BW <valor>: Retrocede.
- LEFT <valor>: Gira a la izquierda si es un número (ej: 90) o desliza a la izquierda si tiene unidad (ej: 2s, 5m).
- RIGHT <valor>: Gira o desliza a la derecha.
- LT <valor>: Sinónimo de LEFT.
- RT <valor>: Sinónimo de RIGHT.
- LOOK <grados>: Mueve la cámara verticalmente.
- JUMP <segundos>: Salta (mantiene el botón).
- INTERACT: Interactúa (un frame).
- WAIT <segundos>: Espera sin input.
- SET <propiedad> <valor>: Fija estado inicial.

- ASSERT <condición> "mensaje": Verifica condición al final.
- SECTION "nombre" ... END: Agrupa acciones y asserts.

26.4. Modificadores de Velocidad

- WALK <acción>: Fuerza a caminar (ej: WALK FW 2s).
- RUN <acción>: Fuerza a correr (ej: RUN LEFT 5m). Por defecto las acciones de movimiento corren.

26.5. Modificadores avanzados

- AT <segundos> <acción>: Ejecuta una acción en un frame específico dentro de otra acción.

26.6. Ejemplo completo

```
// Test de salto y avance
SET pos (0, 0, 0)
SET rot 0

SECTION "Salto y avance"
  FW 1.0
  JUMP 0.2 AT 0.5 FW 0.5 // Salta a los 0.5s mientras avanza
  WAIT 0.3
  ASSERT pos.z > 1.5 "No avanzó lo suficiente"
END
```

27. Especificación: PushableBoxV2 (Objeto Híbrido Determinista)

27.1. Objetivo

Crear un objeto con comportamiento físico dual: un modo dinámico para caídas y rotaciones realistas (6DoF), y un modo cinemático para reposo, apilamiento y transporte determinista. Este sistema permite realismo visual sin comprometer la integridad del replay.

27.2. Clase Base y Configuración

- Nodo: `RigidBody` (configurado para alternar modos).
- Grupo: `replay_sync`.
- Configuración inicial: `mode = MODE_RIGID`. El control de `step(dt)` manejará el cambio a `MODE_KINEMATIC`.

27.3. Lógica de Estados (Híbrido)

27.3.1. Estado Dinámico (Caída/Caos)

Cuando el objeto es golpeado con fuerza o cae de una plataforma:

- Se comporta como un `RigidBody` estándar de Godot.
- Permite rotaciones en los 3 ejes y colisiones complejas.
- Determinismo: para minimizar el drift, forzar `sleeping = false` y registrar cada frame en replay.

27.3.2. Estado Cinemático (Asentamiento)

Cuando la velocidad lineal y angular caen por debajo de un umbral durante N frames:

- Snap de rotación: si está cerca de los 90°, hacer snap a ejes globales para asegurar apilado.
- Cambio de modo: pasar a `MODE_KINEMATIC`.
- Drift correction: redondear posición a 4 decimales para eliminar inconsistencias de coma flotante.

27.4. Integración de Simulación `step(dt)`

En lugar de dejarlo 100% en manos del motor, el `step` supervisa el estado:

- Monitoreo: si está en modo dinámico, verificar `linear_velocity.length() < 0.1`.
- Transición: si se cumple reposo, guardar la posición actual como nueva posición de anclaje determinista.
- Interacción: si el jugador toca el objeto en modo cinemático, “despertar” y volver a modo dinámico.

27.5. Contrato de Replay (Snapshots)

El snapshot debe capturar el modo actual para saber cómo restaurar la física.

```
func get_snapshot() -> Dictionary:
    return {
        "pos": [global_transform.origin.x, global_transform.origin.y, global_transform.origin.z],
        "rot": [global_transform.basis.get_euler().x, global_transform.basis.get_euler().y, global_transform.basis.get_euler().z],
        "vel": [linear_velocity.x, linear_velocity.y, linear_velocity.z],
        "ang": [angular_velocity.x, angular_velocity.y, angular_velocity.z],
        "mode": mode # Captura si es RIGID o KINEMATIC
    }

func restore_snapshot(data: Dictionary):
```

28. Especificación: Refinamiento de Gamefeel y Mecánicas (Core V2)

28.1. Salto Variable (Short Jump)

Para mejorar el control, el jugador debe poder controlar la altura del salto según cuánto tiempo presiona el botón.

- Lógica: si el jugador suelta el botón de salto (`!input.jump`) y la velocidad vertical es mayor a un umbral (ej. `jump_velocity * 0.5`), aplicar frenado o recortar velocidad vertical inmediatamente.
- Determinismo: esta comprobación debe ocurrir exclusivamente dentro de `step(dt, input)` de `PlayerJumpV2`.

28.2. Fricción Diferenciada (Air vs Ground)

El control en el aire debe sentirse más pesado o con menos capacidad de giro para evitar exceso de agilidad.

Nuevas constantes:

- `movement_friction`: usada cuando `is_on_floor()`.
- `air_friction`: usada cuando `!is_on_floor()` (valor menor para más inercia o mayor para restringir control).

Implementación: `PlayerMovementV2` debe seleccionar la fricción basada en el estado de colisión del frame anterior.

28.3. Curvas de Aceleración y Suavizado

Sustituir interpolaciones lineales por curvas de aceleración (easing) para evitar movimiento robótico.

- Plataformas: usar `SINE` o `CUBIC` en los extremos del recorrido.
- Curvas: usar `data/curves` para las aceleraciones.
- Input/movimiento: aplicar curva al `wish_direction`; no alcanzar velocidad máxima instantáneamente.
- Cámara: usar suavizado (`SmoothDamp` o `Lerp` determinista) en `step_camera(dt)` para evitar jitter en replay.

28.4. Re-añadir Tank Turn (Opcional por Zona)

El sistema debe permitir activar modo de giro tipo Tank (estilo clásico):

- Input lateral: rota al personaje sobre eje Y.
- Input vertical: mueve hacia adelante/atrás en su vector frontal (`basis.z`).
- Estado: booleano `use_tank_controls`, conmutable y capturable en snapshot.

28.5. Agachado (Crouch)

- Mecánica: al presionar agachado, reducir altura de `CollisionShape` y penalizar `max_speed` (ej. 50%).
- Visual: `PilotAnimatorV2` debe recibir señal/estado para `crouch_idle_loop` o `crouch_fwd_loop`.
- Importante: reducir la colisión hacia arriba para evitar atravesar suelo o quedar atrapado en techos bajos.

28.6. Reglas de Oro para el Agente

- Snapshots: cualquier variable nueva (`is_crouching`, `current_friction`, etc.) debe incluirse en snapshot.
- Fixed step: prohibido usar delta variable de `_process`; todo refinamiento visual atado a `FIXED_DT`.
- Fricción de conveyors: la velocidad heredada debe sumarse al vector final de movimiento, no sustituirlo.